

Eato



Project ID: Fall-2025-06

Session: BSCS Fall 2022 to 2026

Project Advisor: Salman Irfan

Submitted By

Arham Saleem	70136729
Talha Altaf	70137242

Department of Computer Science & IT
The University of Lahore
Lahore, Pakistan

Declaration

We have read the project guidelines and we understand the meaning of academic dishonesty, in particular plagiarism and collusion. We hereby declare that the work we submitted for our final year project, entitled **Eato** is original work and has not been printed, published or submitted before as final year project, research work, publication or any other documentation.

Group Member 1: Arham Saleem

SAP No: 70136729

Signature:

Group Member 2: Talha Altaf

SAP No: 70137242

Signature:

Statement of Submission

This is to certify that **Arham Saleem** sap id **70136729**, **Talha Altaf** sap id **70137242** have successfully submitted the final project named **Eato**, at Computer Science & IT Department, The University of Lahore, Lahore Pakistan, to fulfill the partial requirement of the degree of **BS in Computer Science**.

Supervisor Name: Salman Irfan

Signature:

Date: June 4, 2026

Dedication

This Final Year Project is respectfully dedicated to our parents, whose unwavering support, sacrifices, and commitment to our education have been the foundation of our academic journey. Their encouragement, patience, and belief in our abilities provided the motivation required to complete this work successfully. We also dedicate this project to our teachers, supervisors, and mentors, whose guidance, professional insight, and constructive feedback played a vital role in its successful completion. Their dedication to academic excellence and critical thinking significantly contributed to our technical and research development. Furthermore, this project reflects the collective effort, collaboration, and professional values cultivated throughout our academic program and marks an important transition from undergraduate study to professional practice. We submit this work with sincere respect and gratitude to all those who contributed, directly or indirectly, to its completion.

Acknowledgement

We would like to express our sincere gratitude to our project supervisor, Sir Salman Irfan, for his continuous guidance, valuable feedback, and professional support throughout the development of this Final Year Project. His expertise, constructive criticism, and encouragement played a crucial role in refining our ideas and ensuring the academic and technical quality of this work. We are also thankful to the faculty members of the Department of Computer Science & IT, The University of Lahore, for providing the theoretical knowledge and practical skills that enabled us to successfully undertake this project. Their commitment to academic excellence and skill-based learning significantly contributed to our professional growth. Additionally, we acknowledge the support and cooperation of our peers for their academic discussions, collaborative efforts, and constructive input during this project. We also extend our sincere appreciation to our families for their patience, understanding, and continuous support, which served as a constant source of motivation throughout our academic journey.

Date: June 4, 2026

Abstract

Eato is a QR-based dine-in food ordering system developed for small scale restaurants seeking an affordable alternative to custom websites and mobile applications. The system addresses challenges associated with traditional ordering methods, such as order inaccuracies, longer waiting times, and operational inefficiencies. Customers can scan a table-specific QR code to access the digital menu, place orders, and communicate requests directly through their smartphones. Restaurant owners manage menus, tables, and orders through an integrated admin panel, while kitchen staff receive real-time order updates via a chef panel. The implementation of Eato improves order accuracy, streamlines restaurant operations, and enhances the overall dining experience. The project provides a scalable and cost-effective digital ordering solution for small food businesses.

Area of the Project

The **EATO** project is primarily situated within the domain of Mobile Application Development, specifically focusing on a cross-platform solution built using React Native to serve both Android and iOS users. It also encompasses Backend Development through the implementation of a Node.js server with Express.js and MongoDB for database management, falling under the areas of Database Systems and Server-Side Programming. Furthermore, the project integrates QR Code Technology for user interaction and Real Time System Design to facilitate immediate order updates and synchronization between customers, kitchen staff, and restaurant administrators. The development process also emphasizes Human Computer Interaction (HCI) and User Experience (UX) Design to ensure an intuitive and efficient interface for all user roles. Collectively, EATO represents a practical application of Full Stack Development within the MERN stack ecosystem, aimed at creating a scalable, digital solution for modern restaurant management.

Technologies used

The EATO project was developed using a modern technology stack comprising React Native for cross-platform mobile application development, Native Wind (Tailwind CSS) for user interface styling, Express.js for backend services, MongoDB for database management, and Figma for UI/UX design and prototyping. These technologies were selected to ensure scalability, maintainability, and a consistent user experience across multiple platforms.

List of Figures

Figure 1 User Interface.....	8
Figure 2 Use case Scan Table QR Code	22
Figure 3 Use case Access Digital menu.....	23
Figure 4 Use case Browse and Search Digital Menu.....	24
Figure 5 Use case Select Items and Add to Cart.....	25
Figure 6 Use case View and Manage Cart.....	26
Figure 7 Use case Place Order	27
Figure 8 Use case Track Order Status.....	28
Figure 9 Use case Owner Login.....	29
Figure 10 Use case Manage menu items.....	30
Figure 11 Use Case View Incoming Orders	31
Figure 12 Use case Update Order Status.....	32
Figure 13 Use case View Order History	33
Figure 14 Architecture Diagram	34
Figure 15 ERD	35
Figure 16 Level 0 DFD	36
Figure 17 Level 1 DFD	37
Figure 18 Class Diagram.....	38
Figure 19 Activity Diagram Scan QR Code	39
Figure 20 Activity Diagram View Digital Menu	39
Figure 21 Activity Diagram Search for Food Items	40
Figure 22 Activity Diagram Add Item to Cart.....	40
Figure 23 Activity Diagram View and Manage Cart.....	41
Figure 24 Activity Diagram Place Order	41
Figure 25 Activity Diagram Track Order Status.....	42
Figure 26 Activity Diagram Owner Login.....	42

Figure 27 Activity Diagram Manage Menu Items	43
Figure 28 Activity Diagram View Incoming Orders	43
Figure 29 Activity Diagram Update Order Status	44
Figure 30 Activity Diagram View Order History	44
Figure 31 Sequence Diagram Scan QR Code	45
Figure 32 Sequence Diagram View Digital Menu.....	45
Figure 33 Sequence Diagram Search for Food Items	46
Figure 34 Sequence Diagram Add Item to Cart.....	46
Figure 35 Sequence Diagram View and Manage Cart.....	47
Figure 36 Sequence Diagram Place Order	47
Figure 37 Sequence Diagram Track Order Status.....	48
Figure 38 Sequence Diagram Owner Login	48
Figure 39 Sequence Diagram Manage Menu Items.....	49
Figure 40 Sequence Diagram View Incoming Orders	49
Figure 41 Sequence Diagram Update Order Status	50
Figure 42 Sequence Diagram View Order History	50
Figure 43 Collaboration Diagram Scan Table QR Code	51
Figure 44 Collaboration Diagram View Digital Menu	51
Figure 45 Collaboration Diagram Search Food Items	51
Figure 46 Collaboration Diagram Add Item to Cart	51
Figure 47 Collaboration Diagram View and Manage Cart	52
Figure 48 Collaboration Diagram Place Order	52
Figure 49 Collaboration Diagram Track Order Status.....	52
Figure 50 Collaboration Diagram Owner Login.....	52
Figure 51 Collaboration Diagram Manage Menu Items	53
Figure 52 Collaboration Diagram View Incoming Orders	53
Figure 53 Collaboration Diagram Update Order Status.....	53
Figure 54 Collaboration Diagram View Order History	53
Figure 55 State Transition Diagram.....	54

Figure 56 Component Diagram.....	55
Figure 57 Deployment Diagram	56
Figure 58 Front Page.....	76
Figure 59 Home page	77
Figure 60 Customer Menu	78
Figure 61 Customer Cart.....	79
Figure 62 Customer Orders Section.....	80
Figure 63 Chef Login.....	81
Figure 64 Chef Dashboard	82
Figure 65 Restaurant Setup	83
Figure 66 OTP Verification	84
Figure 67 Restaurant Details.....	85
Figure 68 Add Categories	86
Figure 69 Add Menu Items	87
Figure 70 Get QR-Codes.....	88
Figure 71 Owner Login.....	89
Figure 72 Owner Dashboard	90
Figure 73 Owner Orders View.....	91
Figure 74 Update Menu	92
Figure 75 Update Profile.....	93

List of Tables

Table 1 Functional Requirement Scan Table QR Code	10
Table 2 Functional Requirement View Digital Menu.....	11
Table 3 Functional Requirement Search for Food Items	11
Table 4 Functional Requirement Add Item to Cart.....	12
Table 5 Functional Requirement View and Manage Cart.....	12
Table 6 Functional Requirement Place Order	13
Table 7 Functional Requirement Track Order Status	13
Table 8 Functional Requirement Owner Login	14
Table 9 Functional Requirement Manage Menu Items.....	14
Table 10 Functional Requirement View Incoming Orders	15
Table 11 Functional Requirement Update Order Status	15
Table 12 Functional Requirement View Order History.....	16
Table 13 Use case Scan Table Code (UC_01)	22
Table 14 Use case View Restaurant Menu (UC_02)	23
Table 15 Use case Browse and Search Digital Menu (UC_03).....	24
Table 16 Use case Select and Add To Cart (UC_04).....	25
Table 17 Use case View and Mange Cart (UC_05).....	26
Table 18 Place order (UC_06)	27
Table 19 Use case Track order status (UC_07)	28
Table 20 Use case Owner Login (UC_08).....	29
Table 21 Use case Manage menu items (UC_09).....	30
Table 22 Use case View Incoming Orders (UC_10)	31
Table 23 Use case Update Order Status (UC_11).....	32
Table 24 Use case View Order History (UC_12)	33
Table 25 Positive test case Menu View Success.....	57
Table 26 Negative Test Case Track Order Failure.....	57

Table 27 Positive Test Case Search Success.....	58
Table 28 Negative Test Case Search Failure	58
Table 29 Positive Test Case Cart Add Success.....	59
Table 30 Negative Test Case Cart Add Failure	60
Table 31 Positive Test Case Order Place success	60
Table 32 Negative Test Case Order Place Faliure	61
Table 33 Positive Test Case Track Order Success.....	61
Table 34 Negative Test Case Track order Success	62
Table 35 Positive Test Case Owner Login Success	63
Table 36 Negative Test Case Owner Login Failure.....	63
Table 37 Positive Test Case Status update Success	64
Table 38 Negative Test Case Status Update Failure.....	65
Table 39 Equivalence Partitions.....	66
Table 40 State Transition testing	66

Table of Contents

<i>Declaration</i>	i
<i>Statement of Submission</i>	ii
<i>Dedication</i>	iii
<i>Acknowledgement</i>	iv
<i>Abstract</i>	v
<i>Area of the Project</i>	vi
<i>Technologies used</i>	vi
List of Figures	vii
List of Tables	x
Chapter 1: Introduction to the Problem	1
1.1 Introduction	1
1.2 Purpose	1
1.3 Objective	2
1.4 Existing Solution	2
1.5 Proposed Solution	3
Chapter 2: Software Requirement Specification	5
2.1 Introduction	5
2.1.1 Purpose	5
2.1.2 Scope	5
2.1.3 Definitions, acronyms, and abbreviations	6
2.2 Overall description	7
2.2.1 Product perspective	7
2.2.2 Product functions	10
2.2.3 User characteristics	16
2.2.4 Constraints	17
2.2.5 Assumptions and dependencies	19
2.2.6 Apportioning of requirements	19
2.3 Specific requirements	20

2.3.1	Functional Requirement.....	20
2.3.2	Non-functional Requirements.....	21
Chapter 3: Use Case Analysis.....		22
Chapter 4: Design		34
4.1	Architecture Diagram	34
4.2	ERD with data dictionary	35
4.3	Data Flow Diagram	36
4.3.1	Level 0	36
4.3.2	Level 1	37
4.4	Class Diagram.....	38
4.5	Activity Diagram.....	39
4.5.1	Activity Diagram AD-01: Scan Table QR Code (FR-01)	39
4.5.2	Activity Diagram AD-02: View Digital Menu (FR-02).....	39
4.5.3	Activity Diagram AD-03: Search for Food Items (FR-03)	40
4.5.4	Activity Diagram AD-04: Add Item to Cart (FR-04).....	40
4.5.5	Activity Diagram AD-05: View and Manage Cart (FR-05).....	41
4.5.6	Activity Diagram AD-06: Place Order (FR-06).....	41
4.5.7	Activity Diagram AD-07: Track Order Status (FR-07).....	42
4.5.8	Activity Diagram AD-08: Owner Login (FR-08).....	42
4.5.9	Activity Diagram AD-09: Manage Menu Items (FR-09).....	43
4.5.10	Activity Diagram AD-10: View Incoming Orders (FR-10)	43
4.5.11	Activity Diagram AD-11: Update Order Status (FR-11)	44
4.5.12	Activity Diagram AD-12: View Order History (FR-12).....	44
4.6	Sequence Diagram.....	45
4.6.1	Sequence Diagram SD-01: Scan Table QR Code (FR-01).....	45
4.6.2	Sequence Diagram SD-02: View Digital Menu (FR-02)	45
4.6.3	Sequence Diagram SD-03: Search Food Items (FR-03)	46
4.6.4	Sequence Diagram SD-04: Add Item to Cart (FR-04)	46
4.6.5	Sequence Diagram SD-05: View and Manage Cart (FR-05)	47
4.6.6	Sequence Diagram SD-06: Place Order (FR-06)	47

4.6.7 Sequence Diagram SD-07: Track Order Status (FR-07)	48
4.6.8 Sequence Diagram SD-08: Owner Login (FR-08)	48
4.6.9 Sequence Diagram SD-09: Manage Menu Items (FR-09)	49
4.6.10 Sequence Diagram SD-10: View Incoming Orders (FR-10).....	49
4.6.11 Sequence Diagram SD-11: Update Order Status (FR-11).....	50
4.6.12 Sequence Diagram SD-12: View Order History (FR-12)	50
4.7 Collaboration Diagram.....	51
4.7.1 Collaboration Diagram CD-01: Scan Table QR Code (FR-01)	51
4.7.2 Collaboration Diagram CD-02: View Digital Menu (FR-02)	51
4.7.3 Collaboration Diagram CD-03: Search Food Items (FR-03)	51
4.7.4 Collaboration Diagram CD-04: Add Item to Cart (FR-04)	51
4.7.5 Collaboration Diagram CD-05: View and Manage Cart (FR-05)	52
4.7.6 Collaboration Diagram CD-06: Place Order (FR-06)	52
4.7.7 Collaboration Diagram CD-07: Track Order Status (FR-07).....	52
4.7.8 Collaboration Diagram CD-08: Owner Login (FR-08).....	52
4.7.9 Collaboration Diagram CD-09: Manage Menu Items (FR-09)	52
4.7.10 Collaboration Diagram CD-10: View Incoming Orders (FR-10)	53
4.7.11 Collaboration Diagram CD-11: Update Order Status (FR-11).....	53
4.7.12 Collaboration Diagram CD-12: View Order History (FR-12)	53
4.8 State Transition Diagram	54
4.9 Component Diagram.....	55
4.10 Deployment Diagram.....	56
Chapter 5: Testing.....	57
5.1 Test Case Specifications.....	57
5.1.1 Test Case for View Digital Menu.....	57
5.1.2 Test Case for Search Food Items	58
5.1.3 Test Case for Add Item to Cart.....	59
5.1.4 Test Case for Place Order	60
5.1.5 Test Case for Track Order Status	61
5.1.6 Test Case for Owner Login	63

5.1.7 Test Case for Update Order Status	64
5.2 Black Box Test Cases	65
5.2.1 Equivalence Partitions (EP).....	66
5.2.2 Boundary Value Analysis	66
5.2.3 Decision Table Testing.....	66
5.2.4 State Transition Testing.....	66
5.2.5 Use Case Testing	67
5.3 White Box Test Cases.....	67
5.3.1 Cyclomatic Complexity	67
5.4 Performance Testing.....	67
5.5 Stress Testing	67
5.6 System Testing	67
5.7 Regression Testing	67
5.7.1 Selecting Regression Tests	67
5.7.2 Regression Testing Steps.....	68
Chapter 6: Tools and Techniques	69
6.1 Introduction	69
6.2 Programming Languages	69
6.3 Development Tools and Applications.....	69
6.4 Frameworks and Libraries.....	70
6.5 State Management Libraries	70
6.6 User Interface and Styling Libraries.....	70
6.7 Additional Libraries and Packages	70
6.8 Summary.....	71
Chapter 7: Summary and Conclusion	72
7.1 Summary.....	72
7.2 Achievement of Objectives.....	73
7.3 Benefits of the Proposed System.....	73
7.3.1 Benefits for Customers	73
7.3.2 Benefits for Restaurant Owners.....	74
7.3.3 Benefits for Kitchen Staff.....	74

7.4 Project Limitations.....	74
7.5 Conclusion.....	74
Chapter 8 User Manual.....	76
8.1 Customer Application	76
8.1.1 Launching the App and Scanning QR Code.....	76
8.1.2 Browsing the Menu	77
8.1.3 Adding Items to Cart	78
8.1.4 Viewing and Confirming the Cart	79
8.1.5 Tracking Order Status.....	80
8.2 Kitchen Panel	81
8.2.1 Login for Kitchen Staff.....	81
8.3.2 Chef Dashboard	82
8.3 Setting Up a New Restaurant.....	83
5.3.1 Owner Signup	83
8.3.2 OTP Verification	84
8.3.3 Enter Restaurant Details	85
8.3.4 Add Categories	86
8.3.5 Set Menu.....	87
8.3.6 Enter Number of Tables	88
8.4 Owner Login and Dashboard	89
8.4.1 Logging in as Owner	89
8.4.2 Owner Dashboard Tab.....	90
8.4.3 Orders Tab	91
8.4.4 Menu Tab.....	92
8.4.5 Profile Tab	93
Chapter 9: Lessons Learnt and Future Work	94
9.1 Lessons Learnt	94
9.2 Future Work.....	94
References	96
Appendix	98

Chapter 1: Introduction to the Problem

1.1 Introduction

The small scale restaurant industry remains hindered by outdated, manual order management systems, relying on paper menus and handwritten orders that cause slow service, inaccuracies, and operational inefficiencies [1]. This problem is exacerbated by the high cost and complexity of existing digital platforms, which are often unsuitable for local, dine-in focused establishments. There is a clear market gap for an affordable, integrated solution that streamlines the ordering process while meeting growing demand for contactless service [2], [3].

The EATO project directly addresses this need by proposing a centralized, QR-based mobile ordering system tailored for small restaurants. By enabling real time digital ordering and kitchen coordination, EATO offers a practical and scalable solution that reduces wait times, minimizes errors, and improves customer experience. Given current technological accessibility and market readiness, implementing EATO presents a feasible and timely opportunity to modernize restaurant operations effectively.

1.2 Purpose

The purpose of developing the EATO project is to create a specialized digital solution that directly addresses the operational and financial challenges faced by small scale restaurants in adopting modern technology. This project is driven by the clear need to bridge the gap between traditional manual service methods and contemporary digital expectations, providing an accessible platform that enhances efficiency, accuracy, and customer satisfaction.

The core need stems from the widespread reliance on paper-based systems [4], which lead to slow service, order errors, and an inability to scale operations. By introducing a unified QR-code based ordering system, EATO enables restaurants to automate the dine in process, reduce dependency on waitstaff, and facilitate real time communication between customers, kitchen staff, and management. This system is designed to be implemented without high cost or technical complexity, making digital transformation achievable for local businesses.

In the market, EATO serves as an affordable alternative to expensive custom-built platforms, empowering restaurant owners to improve service speed, minimize operational costs, and gather valuable sales insights through a simple interface. For society, the project promotes a

more hygienic, contactless dining experience [5], supports the economic sustainability of local food businesses, and encourages the adoption of digital tools in everyday commerce [6]. Ultimately, EATO aims to elevate the standard of service in the food industry while contributing to the broader movement toward a more efficient and digitally inclusive society.

1.3 Objective

To design and develop a fully functional, cross platform mobile application using React Native that allows customers to scan QR codes, view digital menus, place dine in orders, and track order status in real time.

To implement a secure and scalable backend system using Express.js and MongoDB that supports user authentication, menu management, order processing, and real time data synchronization between customer, kitchen, and admin interfaces.

To create an intuitive and accessible user experience for all system stakeholders, including restaurant customers, kitchen staff, and restaurant administrators, ensuring ease of use without requiring technical training.

To successfully deploy and test the integrated system within a real-world environment, demonstrating measurable improvements in order accuracy, service speed, and operational coordination for small scale restaurants.

1.4 Existing Solution

Current digital solutions in the restaurant market, such as Up Menu [7], primarily focus on online food ordering and delivery management. However, competitor analysis reveals these platforms are not designed for the specific needs of dine in restaurants, leading to significant functional gaps. A major limitation is the lack of a dedicated, table specific QR code ordering system for in house customers. Existing platforms do not integrate a seamless scan to order process, forcing restaurants to rely on separate methods for dining in service.

Furthermore, these systems fail to provide distinct interfaces for different users. Customers, kitchen staff, and managers often share the same dashboard, resulting in operational confusion and inefficient workflow. There is also no real-time order tracking visible to customers during their dine in experience, reducing transparency. Additionally, platforms like Up Menu exhibit technical

flaws, such as generating inaccurate or test orders without validation, which undermines reliability for daily operation.

These solutions are typically built for delivery and takeaway models, leaving a clear gap for a focused, affordable, and integrated dine in ordering system. EATO addresses these shortcomings by offering a unified platform centered on QR based ordering, role specific access, and real time coordination, all tailored for the in-restaurant experience.

1.5 Proposed Solution

The proposed solution is EATO, a dedicated mobile application designed specifically to digitize the dine in ordering process for small to mid-sized restaurants that lack the financial resources to develop custom mobile applications. EATO directly addresses the core limitations of existing market solutions by implementing a centralized, QR-code based system that connects customers, kitchen staff, and restaurant managers within a single, cohesive platform [8]. It eliminates the reliance on paper menus and handwritten tickets by enabling customers to scan a unique table QR code to access a digital menu, build their order, and submit it directly to the kitchen in real time.

To tackle the flaws observed in competitors, EATO introduces three distinct, role specific interfaces within one application: a customer facing ordering interface, a streamlined kitchen display panel for chefs, and a comprehensive management dashboard for restaurant owners. This ensures clarity, security, and efficient workflow separation, which is absent in generalized competitor platforms. Furthermore, the system provides live order status tracking, allowing customers to see the progress of their meal and kitchens to manage preparation timelines effectively.

EATO is superior to existing solutions because it is purpose built for the dine in experience, focusing on affordability, ease of use, and practical functionality. It provides a cost-effective alternative for restaurant owners who cannot invest in bespoke software development. Unlike complex platforms, EATO offers a self-service setup that allows any restaurant to create its digital menu, generate table specific QR codes, and begin operations without technical assistance or high upfront costs. By providing a reliable, integrated, and user centric solution that directly resolves

the operational inefficiencies of traditional and outdated digital systems, EATO presents a more accessible, efficient, and suitable alternative for modernizing restaurant service.

Chapter 2: Software Requirement Specification

2.1 Introduction

2.1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to provide a comprehensive, structured description of the functional and non-functional requirements for the EATO mobile application [9]. This document serves as a formal agreement between stakeholders, including project developers, academic supervisors, and potential end-users, regarding the system's intended capabilities, constraints, and behavior. It aims to establish a clear foundation for design, development, testing, and validation phases, ensuring that the final product aligns with the identified needs of small-scale restaurants and their customers.

The intended audience for this SRS includes software developers, UI/UX designers, quality assurance testers, project supervisors, and restaurant owners or representatives who will participate in acceptance testing. Additionally, this document may be referenced by academic evaluators to assess the project's scope, feasibility, and alignment with initial objectives.

2.1.2 Scope

The software products to be produced under this project are collectively named Eato and consist of the following integrated components: the Eato Customer Mobile Application, the Eato Restaurant Management Interface, the Eato Kitchen Panel, and the Backend Services and Database System.

The Eato Customer Mobile Application will allow dine-in customers to scan a table-specific QR code, view a digital menu, select items, place an order, and track its status in real time. It will not support ordering from outside the restaurant, online payments, or delivery service integration. The Eato Restaurant Management Interface will enable restaurant owners to manage their menu, table setup, and QR codes, and to view all active orders. It will not provide advanced accounting, inventory management, or payroll functionalities. The Eato Kitchen Panel will give kitchen staff a real-time view of incoming orders and allow them to update order statuses. It will not handle supplier management or recipe costing. The Backend System will manage data storage, user authentication, and real-time synchronization; it will not interface with third-party delivery or payment gateways.

The application of this software is to provide a unified, affordable digital solution for small-scale restaurants to digitize their dine-in ordering process. Its primary benefit is replacing error-prone paper-based systems with a streamlined digital workflow, leading to faster service, reduced operational errors, and enhanced customer experience through transparency and reduced wait times. The key objective is to make basic digital transformation accessible and manageable for restaurants without the resources for custom app development.

This defined scope is consistent with the system requirements and objectives stated in the higher-level project proposal, ensuring the final product remains focused on its core purpose of modernizing in-restaurant service.

2.1.3 Definitions, acronyms, and abbreviations

This section provides detailed definitions, acronyms, and abbreviations used throughout this Software Requirements Specification document. Its purpose is to ensure clear, consistent, and unambiguous interpretation of the terms and concepts related to the Eato system by all stakeholders, including evaluators, developers, and academic reviewers.

The term Eato refers to a centralized QR based dine in food ordering system designed to enable interaction between customers, restaurant owners, and kitchen staff through a single integrated mobile application [2]. A dine in ordering system is defined as a software system that allows customers to place food orders while physically present inside a restaurant without involving delivery or takeaway services. A QR code is a two-dimensional machine-readable code that encodes restaurant specific and table specific information and is used to uniquely identify a customer's dining location within the system. A table specific QR code is assigned to an individual restaurant table and links customer orders to that table to ensure accurate order routing and tracking.

A digital menu is an electronic representation of a restaurant's menu that includes food categories, item names, descriptions, and prices, displayed within the application interface [3]. The customer interface refers to the part of the application through which customers scan QR codes, browse the menu, place orders, and view order status. The management interface is an embedded interface within the application that allows restaurant owners or administrators to manage menu items, food categories, table configuration, and incoming orders. The chef panel is a dedicated

interface designed for kitchen staff to receive real time order notifications, view order details, and update order preparation status.

An order is defined as a collection of food items selected by a customer and submitted through the application for preparation by the kitchen. Order status represents the current processing stage of an order, such as pending, preparing, or ready, as updated by kitchen staff [10]. The order lifecycle refers to the complete sequence of states an order passes through from placement to completion. Real time processing is the capability of the system to immediately synchronize data changes across all relevant interfaces without requiring manual refresh.

Role based access control is a security mechanism that restricts system functionality based on predefined user roles such as customer, restaurant owner, or kitchen staff [11]. Authentication is the process of verifying the identity of a user before granting access to restricted areas of the application, while authorization determines which actions or resources a verified user is permitted to access [12]. Backend services refer to cloud-based services responsible for data storage, authentication, and real time communication between different components of the application. A database is a structured data storage system used to store information related to users, menus, tables, and orders. System scalability is defined as the ability of the system to handle increasing numbers of users and orders without degradation in performance.

The acronyms used in this document include SRS for Software Requirements Specification, QR for Quick Response, UI for User Interface, UX for User Experience, API for Application Programming Interface, CRUD for Create, Read, Update, Delete, REST for Representational State Transfer [13], and DBMS for Database Management System. The abbreviations used include UOL for University of Lahore, App for Application, DB for Database, ID for Identifier, Auth for Authentication, and OS for Operating System.

2.2 Overall description

2.2.1 Product perspective

The Eato system is an independent and self contained mobile based dine in food ordering application developed for small scale restaurants. The system is designed to operate entirely within a single mobile application environment and does not rely on external web-based platforms or third-party ordering services. All core functionalities, including customer ordering, restaurant

management, and kitchen operations, are integrated within the application and supported by cloud-based backend services for real time data synchronization and storage.

From a product perspective, Eato is composed of multiple logical components that serve different user roles. These components include a customer facing ordering interface, a restaurant owner or manager interface, and a kitchen or chef interface. All users initially interact with a common entry screen, which serves as the primary access point to the system. Based on user actions and authentication, the system navigates users to role specific interfaces.

System Interfaces:

The system interfaces describe how the Eato application interacts with various system-level services to ensure smooth and reliable operation. The application communicates with backend services to perform core functions such as QR code validation, menu retrieval, order placement, order status updates, and user authentication. These backend interfaces allow the system to process user requests and manage data efficiently.

The QR scanning functionality interfaces directly with the mobile device camera to scan and interpret the QR code, which is used to identify the restaurant and the corresponding table number. Additionally, the system interfaces with network services to enable real-time communication between customers, restaurant management, and kitchen staff, ensuring that orders and status updates are synchronized across all parts of the application.

User Interfaces:

The user interface of Eato is designed to be intuitive, visually consistent, and suitable for real world restaurant environments. The initial interface presented to all users includes a QR code scanning screen, where customers are guided to scan the table specific QR code to begin the ordering process. The screen provides clear visual boundaries for scanning, instructional text, and optional demo scan buttons for testing purposes. A dedicated owner login option is also available on layouts that include images, item names, brief descriptions, prices,



Figure 1 User Interface

this screen to allow restaurant owners or managers to access administrative functionality. After successful QR scanning, customers are directed to the restaurant home screen, which displays the restaurant name, table number, rating, and featured food items. Food items are presented in card-based the cart. A bottom navigation bar is provided to allow easy access to key sections of the application, including Home, Menu, Cart, and Orders.

The menu interface allows customers to search for food items using a search bar and browse available menu options. Clear feedback messages are displayed when no matching results are found. The consistent layout and navigation design across screens ensures ease of use and reduces user confusion. Additional interfaces for cart management, order tracking, restaurant management, and kitchen operations are part of the system and are either under development or accessible based on user roles.

Hardware Interfaces:

The Eato application depends on standard smartphone hardware components. These include the device camera for QR code scanning, touchscreen input for user interaction, and network hardware for internet connectivity. No specialized hardware devices are required, making the system suitable for deployment in every restaurant environment.

Software Interfaces:

The software interfaces include interaction with the mobile operating system for camera access, permissions management, and network communication. The application interfaces with cloud-based backend services and databases for persistent data storage, authentication, and real time order synchronization. All software components operate within the mobile application without the need for external web-based systems.

Communications Interfaces:

The system uses internet-based communication to exchange data between the mobile app and backend services. Standard network protocols support QR verification, menu loading, order placement, and status updates within the restaurant's local network.

Memory:

The memory requirements of Eato are compatible with modern mobile devices. Primary memory is used to manage active sessions, user interface rendering, menu data, and order

information. Secondary memory may be used for temporary caching to improve performance. Long term data storage is handled by cloud-based databases to ensure reliability and scalability.

Operations:

Normal operations of the system include QR code scanning, menu browsing, adding items to the cart, placing dine in orders, and tracking order status. Administrative operations include menu management and order monitoring by restaurant owners. Special operations such as data backup and recovery are managed through backend services to ensure data integrity and availability.

Site Adaptation Requirements:

The Eato system requires minimal site-specific configuration. Each restaurant must register within the system, configure menu items and prices, define table information, and generate table specific QR codes. QR codes must be placed on restaurant tables, and stable network connectivity must be available within the restaurant. No additional hardware or complex customization is required.

2.2.2 Product functions

The Eato system includes key functional requirements for QR-based ordering, restaurant management, and kitchen operations, with defined inputs, outputs, and workflows.

Table 1 Functional Requirement Scan Table QR Code

Field	Description
ID	FR-01
Name	Scan Table QR Code
Description	The system allows customers to scan a table specific QR code to identify the restaurant and table number before accessing the digital menu.
Input	Table specific QR code captured through the mobile device camera.
Output	Successful identification of restaurant and table number, followed by redirection to the restaurant home screen.
Basic Workflow	The user opens the app and scans the table QR code. The system validates the QR code and loads the restaurant home screen.

Table 2 Functional Requirement View Digital Menu

Field	Description
ID	FR-02
Name	View Digital Menu
Description	The system displays the restaurant's digital menu, including food items, descriptions, prices, and images.
Input	Validated restaurant and table information.
Output	A list of available food items displayed in a card based layout.
Basic Workflow	The system retrieves menu data from the backend. Food categories and items are displayed on the home and menu screens. The user scrolls through available items.

Table 3 Functional Requirement Search for Food Items

Field	Description
ID	FR-03
Name	Search for Food Items
Description	The system allows customers to search for food items using a keyword-based search.
Input	Text entered in the search bar.
Output	Filtered list of food items or a no results message.
Basic Workflow	The user enters a keyword in the search field. The system processes the input. The matching food items are displayed. If no match is found, an appropriate message is shown.

Table 4 Functional Requirement Add Item to Cart

Field	Description
ID	FR-04
Name	Add Item to Cart
Description	The system allows customers to add selected food items to the shopping cart.
Input	Food item selection and quantity.
Output	Updated cart containing selected items.
Basic Workflow	The user selects a food item by taping the add to cart button. The system adds the item to the cart. The cart icon updates accordingly

Table 5 Functional Requirement View and Manage Cart

Field	Description
ID	FR-05
Name	View and Manage Cart
Description	The system allows customers to view cart contents and manage item quantities.
Input	User actions such as increase, decrease, or remove item.
Output	Updated cart summary and total amount.
Basic Workflow	The user opens the cart screen. The system displays selected items. The user modifies quantities or removes items. The system updates the cart in real time.

Table 6 Functional Requirement Place Order

Field	Description
ID	FR-06
Name	Place Order
Description	The system allows customers to place a dine in food order after finalizing cart items.
Input	Confirmed cart data and table information.
Output	Order submission confirmation.
Basic Workflow	The user reviews the cart. The user confirms the order. The system sends the order to the kitchen. The system displays order confirmation.

Table 7 Functional Requirement Track Order Status

Field	Description
ID	FR-07
Name	Track Order Status
Description	The system allows customers to view the real time status of their order.
Input	Order identifier.
Output	Current order status displayed to the customer.
Basic Workflow	The system receives status updates from the kitchen. The customer views the order status screen. Updated status is displayed in real time.

Table 8 Functional Requirement Owner Login

Field	Description
ID	FR-08
Name	Owner Login
Description	The system allows restaurant owners or managers to log in to access management features.
Input	Owner credentials.
Output	Authenticated access to management interface.
Basic Workflow	The owner selects the login option. The owner enters credentials. The system validates credentials. Access to management interface is granted.

Table 9 Functional Requirement Manage Menu Items

Field	Description
ID	FR-09
Name	Manage Menu Items
Description	The system allows restaurant owners to create, update, or remove menu items.
Input	Food item details such as name, price, description, and image.
Output	Updated digital menu.
Basic Workflow	The owner accesses menu management. The owner adds or edits food items. The system saves changes. The updated menu is displayed to customers.

Table 10 Functional Requirement View Incoming Orders

Field	Description
ID	FR-10
Name	View Incoming Orders
Description	The system allows kitchen staff to view new and pending orders.
Input	Order data from customers.
Output	List of incoming orders displayed on chef panel.
Basic Workflow	The system receives a new order. The order appears on the chef panel. Kitchen staff review order details.

Table 11 Functional Requirement Update Order Status

Field	Description
ID	FR-11
Name	Update Order Status
Description	The system allows kitchen staff to update the preparation status of orders.
Input	Selected order and status update.
Output	Updated order status visible to customer and management.
Basic Workflow	Kitchen staff selects an order. Status is updated to preparing or ready. The system synchronizes status across interfaces.

Table 12 Functional Requirement View Order History

Field	Description
ID	FR-12
Name	View Order History
Description	The system allows restaurant owners to view completed and past orders.
Input	Owner request for order records.
Output	Displayed list of previous orders.
Basic Workflow	The owner opens order history. The system retrieves stored data. Orders are displayed chronologically.

2.2.3 User characteristics

The Eato system is designed to be used by multiple categories of users with varying levels of technical expertise. The primary user groups of the system include customers, restaurant owners or managers, and kitchen staff. The application is intentionally designed to accommodate users with minimal technical knowledge while maintaining sufficient functionality for administrative and operational tasks.

Customers are general restaurant visitors with diverse educational backgrounds and varying levels of technical experience. It is assumed that customers possess basic smartphone usage skills, such as operating a touch screen, scanning a QR code, and navigating simple application interfaces. No prior training or technical expertise is required for customers to use the system, as the user interface is designed to be intuitive and self explanatory.

Restaurant owners or managers are responsible for managing menus, tables, and monitoring orders through the application. These users are expected to have a basic understanding of smartphone applications and general digital concepts. While advanced technical skills are not required, owners or managers should be familiar with data entry tasks such as adding menu items, updating prices, and reviewing order information. Minimal training may be sufficient for effective use of management features.

Kitchen staff or chefs interact with the system through the Chef Portal to view and manage incoming orders. These users are not required to have advanced educational qualifications or technical expertise. Basic familiarity with using a mobile device and understanding simple on-screen instructions is sufficient. The chef interface is designed to support quick interactions in a busy kitchen environment with minimal user input.

Overall, the Eato system is designed to be accessible to users with limited technical knowledge while ensuring reliable and efficient operation for all user roles. The system emphasizes usability, clarity, and ease of navigation to support smooth adoption in real world restaurant environments.

2.2.4 Constraints

The Eato system is subject to technical, operational, and regulatory constraints that influence its design and implementation to ensure reliability, security, and compliance.

Regulatory Policies:

The system must comply with general data protection and privacy practices applicable to user information. While no formal regulatory certifications are required for academic deployment, the application is designed to handle user and order data responsibly, ensuring confidentiality and controlled access.

Hardware Limitations:

The Eato application relies on standard smartphone hardware, including a camera for QR scanning, internet connectivity, and adequate processing power for smooth performance. Device specifications may affect performance.

Interfaces to Other Applications:

The system operates as a standalone application and does not integrate with external web platforms, third party delivery services, or payment gateways. This constraint simplifies system design but limits interoperability with external restaurant management systems.

Parallel Operation:

The system must support parallel operation by multiple users, including customers placing orders simultaneously and kitchen staff processing multiple orders at the same time. This requires reliable backend synchronization to prevent data conflicts or order duplication.

Audit Functions:

Basic audit functionality is limited to order history and status tracking within the application. Advanced audit logging and reporting mechanisms are outside the scope of this project and are not implemented.

Control Functions:

Administrative control functions are restricted to authorized users such as restaurant owners and kitchen staff. Access to management and chef interfaces is controlled through authentication mechanisms to prevent unauthorized usage.

Higher Order Language Requirements:

The system is developed using high level programming languages suitable for mobile application development. The choice of language and framework imposes constraints related to platform compatibility and performance optimization.

Signal Handshake Protocols:

Communication between the application and backend services relies on standard internet communication protocols. No custom signal handshake protocols are implemented beyond standard request and response mechanisms.

Reliability Requirements:

The system is required to maintain reliable operation during restaurant hours. Temporary network failures may affect real time updates; however, the system is designed to handle such situations gracefully through backend data consistency mechanisms.

Criticality of the Application:

The Eato system is classified as a non critical application. Failure of the system does not pose a threat to human safety but may temporarily impact restaurant operations and customer experience.

Safety and Security Considerations:

The system includes basic security measures such as authenticated access for management and kitchen interfaces and secure handling of user data. Physical safety risks are minimal, and the application does not control any hardware that could pose safety hazards.

2.2.5 Assumptions and dependencies

The requirements specified in this Software Requirements Specification are based on certain assumptions and external dependencies that may influence the behaviour, performance, and applicability of the Eato system. Any changes to these factors may affect the stated requirements.

It is assumed that users of the system, including customers, restaurant owners, and kitchen staff, have access to smartphones capable of running the application and possess basic skills to operate mobile applications. It is also assumed that customers will have access to the restaurant's QR codes and will be physically present inside the restaurant while placing orders.

The system depends on stable internet connectivity within the restaurant premises for real time communication between the application and backend services. Any disruption in network availability may impact order placement, status updates, and synchronization between different user interfaces.

The Eato system assumes that restaurant owners will correctly configure menu items, prices, and table information during initial setup. Incorrect or incomplete configuration may affect the accuracy of orders and system functionality.

The requirements also depend on the availability and reliability of cloud-based backend services used for authentication, data storage, and real time synchronization. Changes in backend service availability or performance may influence system responsiveness and reliability.

2.2.6 Apportioning of requirements

This subsection identifies system requirements that are not included in the current version of the Eato system but may be considered for future versions. These requirements are excluded from the present scope to maintain focus on core dine in ordering functionality.

Future versions of the Eato system may include online payment functionality to allow customers to complete transactions digitally within the application. Enhanced reporting and analytics feature for restaurant owners, such as detailed order summaries and sales insights, may also be considered in later iterations.

Integration with external restaurant management systems or inventory tracking solutions may be explored in future versions if required by operational needs. Support for takeaway and delivery ordering is outside the scope of the current system and is not implemented in this version.

The apportioning of these requirements ensures that the current version of Eato remains focused on providing a simple QR based dine in food ordering experience without customer account creation, while allowing controlled expansion if required in the future.

2.3 Specific requirements

This section describes the functional and non-functional requirements of the Eato system at a level of detail sufficient for designers to implement the system and for testers to verify its compliance with the stated requirements. These requirements are derived directly from the system scope, user interface, and intended operational behavior.

2.3.1 Functional Requirement

The functional requirements of the Eato system define the actions and services the system must provide to users. The system consists of the following modules:

QR Code Scanning:

Allows customers to scan a table specific QR code to identify the restaurant and table number and access the digital menu [2].

Digital Menu Display:

Presents food categories and items with details including names, descriptions, images, and prices [3].

Search Menu Items:

Enables customers to efficiently search and browse menu items using keywords, helping them quickly find desired food options.

Add Items to Cart:

Allows customers to select food items and add them to their cart for easy order preparation and selection.

Cart Management:

Provides full control to view, update quantities, or remove items from the cart before finalizing the order.

Place Order:

Sends the finalized customer order directly to the kitchen in real time for immediate preparation and processing.

Order Status Tracking:

Provides real-time updates on order preparation status and completion, ensuring users can track progress instantly from kitchen processing to final delivery or pickup.

Owner/Manager Functions:

Allows restaurant owners to manage menus, categories, table information, and incoming orders.

Chef Portal Functions:

Allows kitchen staff to view incoming orders, accept orders, update preparation status, and mark orders as ready.

2.3.2 Non-functional Requirements

The non-functional requirements specify how the system performs and the quality attributes it must maintain. For the Eato system, the following categories apply:

Usability:

The system must be intuitive and easy to navigate for customers, restaurant owners, and kitchen staff. Minimal training is required for each user role. The interfaces use clear instructions, visual indicators, and consistent navigation to support user understanding.

Reliability:

The system must provide consistent performance with minimal downtime. Backend services and cloud infrastructure ensure real time synchronization and data integrity across multiple users and devices.

Performance:

The system must respond to user actions such as QR scanning, menu loading, order placement, and status updates within two seconds under normal network conditions, while maintaining responsiveness during multiple simultaneous orders.

Design Constraints:

The system is designed for mobile platforms (Android and iOS) using a single application environment. The application depends on standard smartphone hardware and cloud-based backend services for data storage and synchronization.

Chapter 3: Use Case Analysis

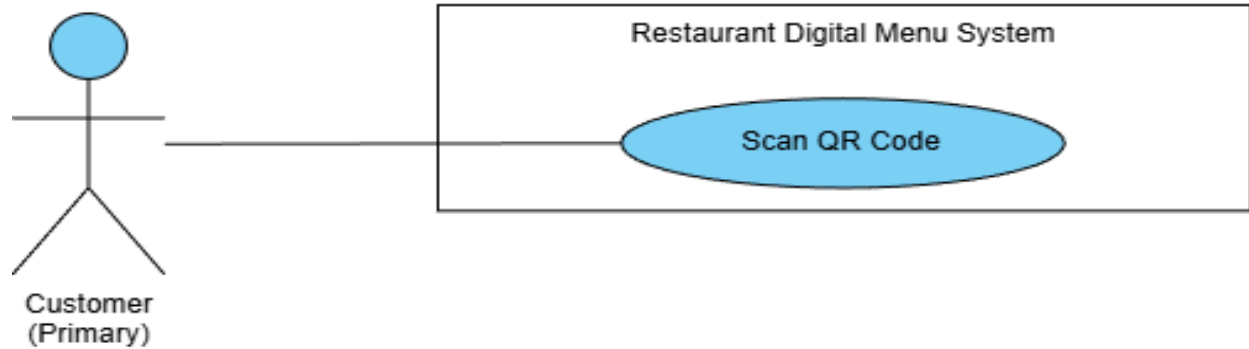


Figure 2 Use case Scan Table QR Code

Table 13 Use case Scan Table Code (UC_01)

Use Case ID	UC_01	
Use Case Name	Scan Table QR Code	
Description	Customer scans a Table Specific QR Code to access the restaurant's Digital Menu.	
Primary Actor	Customer	
Secondary Actor	None	
Pre-Condition	Eato app is installed. Valid Table Specific QR Code is present. Device has camera and internet.	
Post-Condition	Customer views the Digital Menu. System links session to the table.	
Basic Flow	Actor Action	System Action
	• Opens app and selects QR scan. • Points camera at table QR code. • Confirms scan.	• Activates camera. • Decodes QR and validates identifiers. • Fetches and displays Digital Menu. • Logs table session.
Alternate Flow	Manual table code entry.	

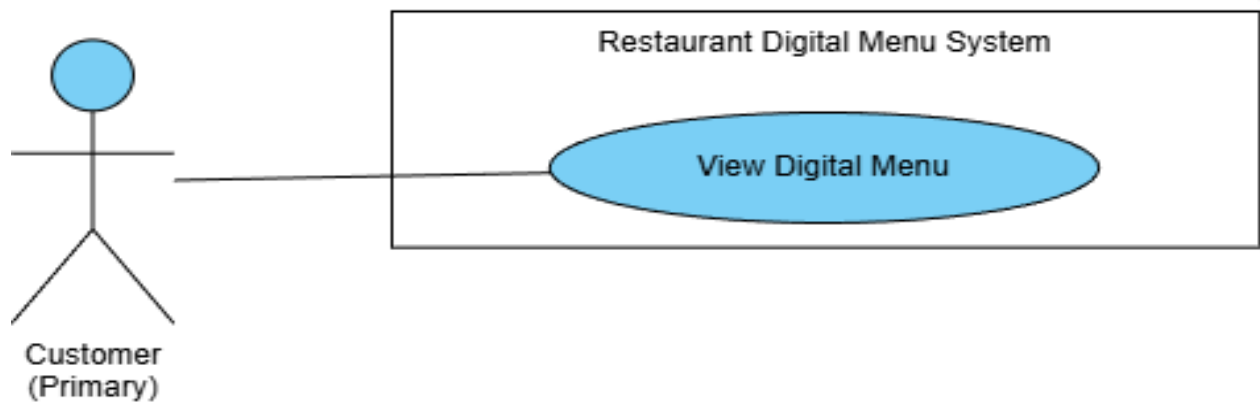


Figure 3 Use case Access Digital menu

Table 14 Use case View Restaurant Menu (UC_02)

Use Case ID	UC_02	
Use Case Name	View Restaurant Menu	
Description	Customer browses the restaurant's Digital Menu loaded after a successful QR scan.	
Primary Actor	Customer	
Secondary Actor	None	
Pre-Condition	QR code has been successfully scanned (UC-01).	
Post-Condition	Digital Menu is displayed to the Customer.	
Basic Flow	Actor Action	System Action
	- Views menu categories. - Selects a category. - Checks item details.	- Displays organized menu. - Shows items with details. - Displays full description.
Alternate Flow	Refresh or reload menu if data does not load initially.	

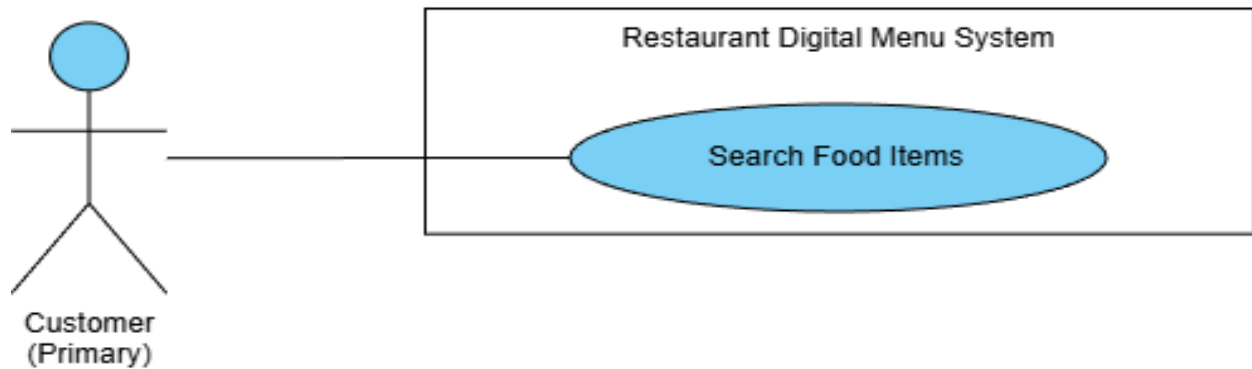


Figure 4 Use case Browse and Search Digital Menu

Table 15 Use case Browse and Search Digital Menu (UC_03)

Use Case ID	UC_03	
Use Case Name	Browse and Search Digital Menu	
Description	After scanning a table QR code, the Customer views the restaurant's Digital Menu and can search for specific food items	
Primary Actor	Customer	
Secondary Actor	None	
Pre-Condition	Customer is seated at a table with a valid QR code and has the Eato app.	
Post-Condition	Customer can view menu items and use the search function to find specific dishes.	
Basic Flow	Actor Action	System Action
	- Views the displayed menu categories and items. - Scroll to look for specific items to order	- Validates code and loads the restaurant's Digital Menu. - Filters and displays matching menu items in real-time. - Filters and displays matching menu items in real-time.
Alternate Flow	Uses the search bar to type a food item name.	

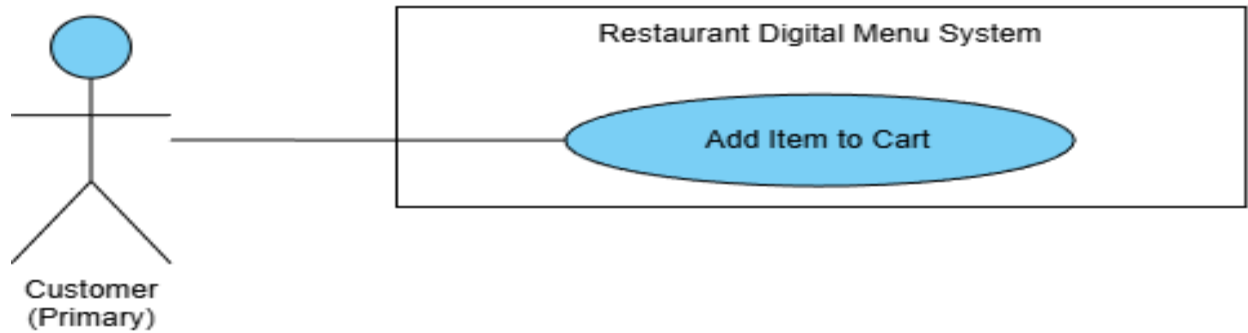


Figure 5 Use case Select Items and Add to Cart

Table 16 Use case Select and Add To Cart (UC_04)

Use Case ID	UC_04	
Use Case Name	Select Items and Add to Cart	
Description	After accessing the Digital Menu, the Customer can browse or search for food items and add their selections to a digital cart for later checkout.	
Primary Actor	Customer	
Secondary Actor	None	
Pre-Condition	Customer has successfully scanned the Table Specific QR Code and the Digital Menu is loaded (UC-01, UC-03).	
Post-Condition	Selected items are added to the customer's cart, and the cart is updated with a running total.	
Basic Flow	Actor Action	System Action
	• Selects a desired item and chooses “Add to Cart.” • Views the cart summary to review selections.	• Displays menu items or search results. • Adds the item to the customer’s active cart session and updates the cart total. • Displays the cart contents with itemized pricing and total.
Alternate Flow	Refresh or reload menu if data does not load initially.	

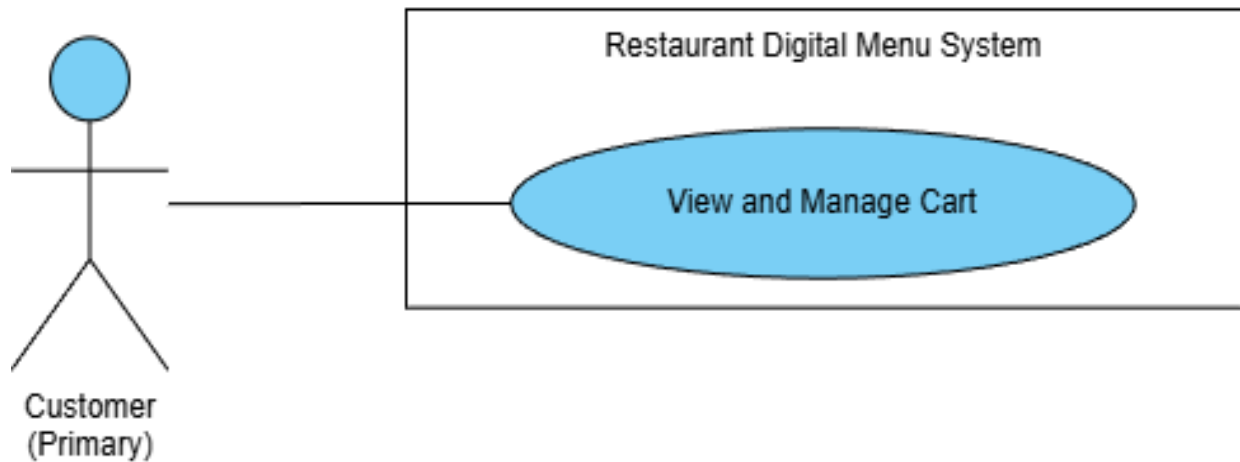


Figure 6 Use case View and Manage Cart

Table 17 Use case View and Mange Cart (UC_05)

Use Case ID	UC_05	
Use Case Name	View and Manage Cart	
Description	The Customer can review the items in their cart, adjust quantities, remove selections, and view the updated total before proceeding to place an order.	
Primary Actor	Customer	
Secondary Actor	None	
Pre-Condition	The Customer has added at least one item to their cart (UC-05).	
Post-Condition	The cart contents and final total are confirmed and ready for order submission.	
Basic Flow	Actor Action	System Action
	• Navigates to view their cart. • Increases or decreases the quantity of an item. • Removes an unwanted item from the cart. • Reviews the final list and total price.	• Displays all cart items with individual prices, quantities, and a subtotal. • Shows a summary and prepares for the checkout step.
Alternate Flow	If the cart is found to be empty, the system will display a message and redirect the customer back to the Digital Menu.	

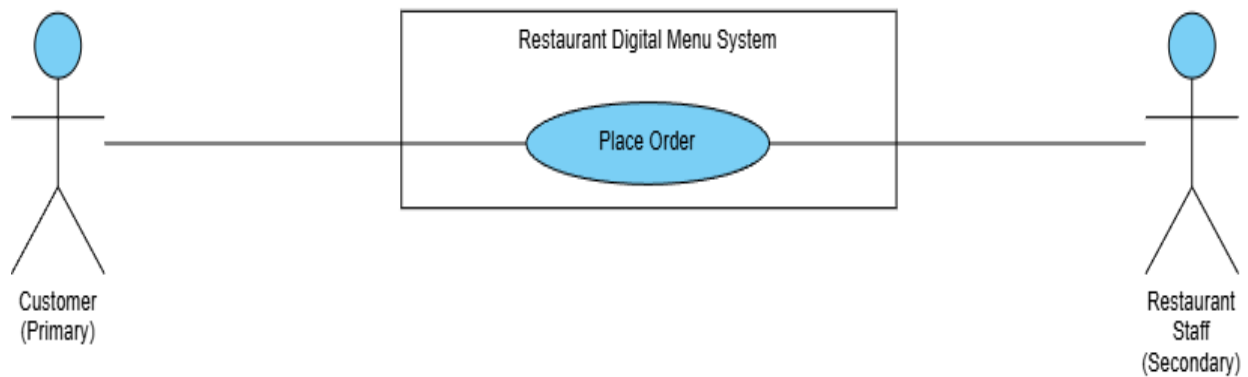


Figure 7 Use case Place Order

Table 18 Place order (UC_06)

Use Case ID	UC_06	
Use Case Name	Place Order	
Description	Customer selects items from the menu and submits an order to the kitchen.	
Primary Actor	Customer	
Secondary Actor	Restaurant Staff (Kitchen)	
Pre-Condition	Customer is viewing the Digital Menu (UC-03).	
Post-Condition	Order is created and sent to the kitchen's Chef Panel.	
Basic Flow	Actor Action	System Action
	- After reviewing the cart, selects “Place Order.” - Confirms the intent to submit.	- Performs a final validation of all items and prices. - Sends the complete order to the kitchen’s Chef Panel in real time, saves it in the Database with a unique Order ID, and clears the cart. - Displays an order confirmation screen with the Order ID, summary, and estimated preparation time.
Alternate Flow	Modify or cancel order before kitchen confirmation	

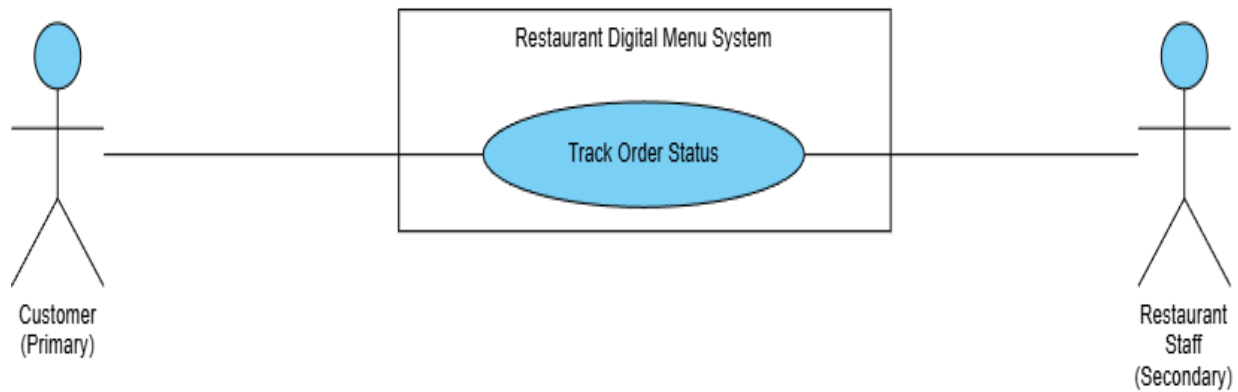


Figure 8 Use case Track Order Status

Table 19 Use case Track order status (UC_07)

Use Case ID	UC_07	
Use Case Name	Track Order Status	
Description	Customer views the real-time status (e.g., Preparing, Ready) of their placed order.	
Primary Actor	Customer	
Secondary Actor	Restaurant Staff (Kitchen)	
Pre-Condition	An order has been placed (UC-06).	
Post-Condition	Current order status is displayed to the Customer.	
Basic Flow	Actor Action	System Action
	• Navigates to the “Track Order” section of the app. • Views the status updates and estimated time.	• Fetches and displays the live status of the customer’s active order using the Order ID. • Continuously polls or receives real-time updates from the kitchen’s Chef Panel and refreshes the display.
Alternate Flow	Request staff assistance if status is unclear or delayed.	

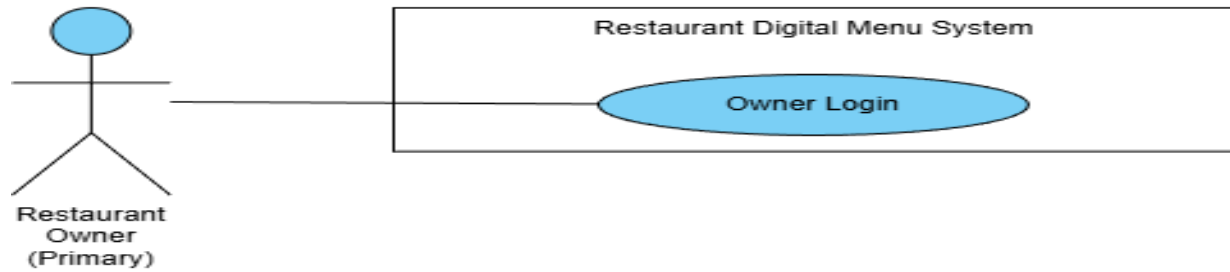


Figure 9 Use case Owner Login

Table 20 Use case Owner Login (UC_08)

Use Case ID	UC_08	
Use Case Name	Owner Login	
Description	This case enables the restaurant owner or manager to authenticate into the system and access the restricted Management Interface for administrative tasks, such as menu management and order oversight.	
Primary Actor	Restaurant Owner	
Secondary Actor	None	
Pre-Condition	The owner has a registered account with valid credentials in the system.	
Post-Condition	The owner is granted authenticated access to the Management Interface.	
Basic Flow	Actor Action	System Action
	- Owner selects the “Login” option and chooses the “Owner” role. - Owner enters registered email and password and submits the form.	- System displays the login form requesting email and password. - System validates the credentials against the Database. - Upon successful validation, the system grants access and redirects to the Management Interface dashboard.
Alternate Flow	If the entered credentials are invalid (e.g. wrong password or unregistered email), the system displays an error message and returns the user to the login form without granting access.	

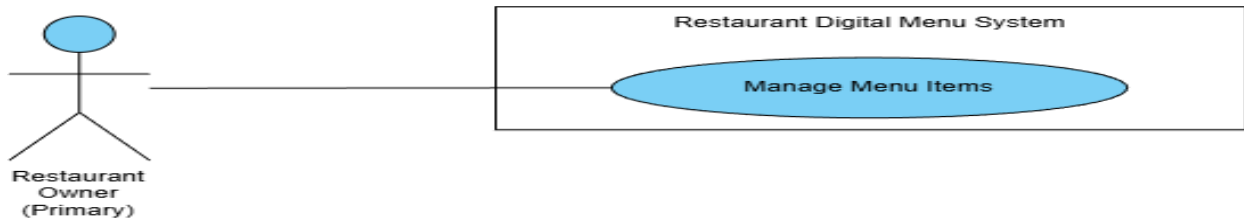


Figure 10 Use case Manage menu items

Table 21 Use case Manage menu items (UC_09)

Use Case ID	UC_09	
Use Case Name	Manage menu items	
Description	This use case allows the authenticated Restaurant Owner to create new menu items, edit existing ones, or remove outdated items from their restaurant's Digital Menu.	
Primary Actor	Restaurant Owner	
Secondary Actor	None	
Pre-Condition	The owner is logged into the Management Interface (UC-08).	
Post-Condition	The Digital Menu is updated and reflects all changes for customers to view.	
Basic Flow	Actor Action	System Action
	- Owner navigates to the "Menu Management" section. - Owner chooses to add a new item or edit/remove an existing one. - Owner enters or modifies the item details and saves changes.	- System displays the current list of menu items. - System provides the appropriate form (with fields for name, price, description, image). - System validates the input and updates the Database. - System confirms the update and refreshes the menu list. The change is reflected in the Customer Interface.
Alternate Flow	If invalid data (e.g., a negative price) is entered, the system rejects the change and prompts the owner to correct it.	

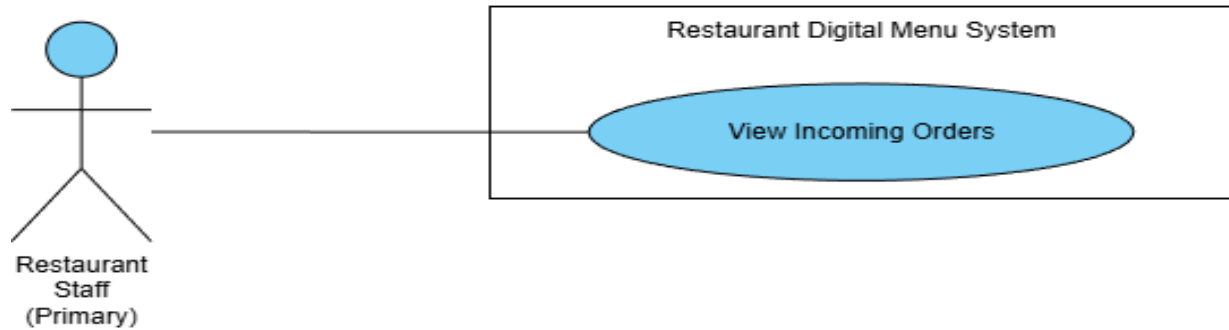


Figure 11 Use Case View Incoming Orders

Table 22 Use case View Incoming Orders (UC_10)

Use Case ID	UC_10	
Use Case Name	View Incoming Orders	
Description	This use case allows kitchen staff to view a list of new and pending orders in real-time through the Chef Panel after a customer has placed an order.	
Primary Actor	Restaurant staff	
Secondary Actor	None	
Pre-Condition	The kitchen staff is logged into the Chef Panel. At least one order has been placed by a customer	
Post-Condition	The kitchen staff has reviewed the incoming order details and can proceed to update their status.	
Basic Flow	Actor Action	System Action
	- Kitchen staff accesses the Chef Panel interface. - Kitchen staff selects an order to view more details.	- System retrieves and displays all incoming orders (new and pending) in a list with details (Order ID, table number, items, quantities, special notes). - System expands the order view to show item details, preparation notes, and timestamp.
Alternate Flow	If there are no incoming orders, the Chef Panel displays a “No new orders” message.	

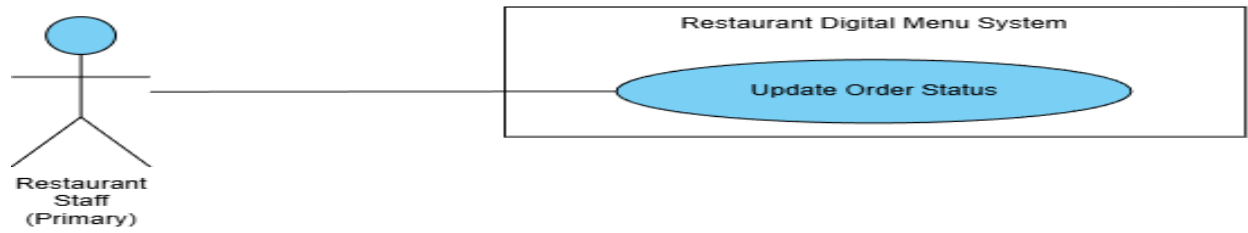


Figure 12 Use case Update Order Status

Table 23 Use case Update Order Status (UC_11)

Use Case ID	UC_11	
Use Case Name	Update Order Status	
Description	This use case allows kitchen staff to change the preparation status of an incoming order (e.g., from "Received" to "Preparing" to "Ready").	
Primary Actor	Restaurant Staff	
Secondary Actor	None	
Pre-Condition	An incoming order is visible on the Chef Panel (UC-11). The kitchen staff is logged in.	
Post-Condition	The order status is updated, and the change is reflected in real-time on both the Customer Interface and the Management Interface.	
Basic Flow	Actor Action	System Action
	Kitchen staff selects an incoming order from the Chef Panel list.	- System displays the current order details and available status options (e.g., "Preparing," "Ready"). - System updates the order's status in the Database. - System notifies the Customer Interface and Management Interface of the change in real-time. - System confirms the update on the Chef Panel.
Alternate Flow	If an invalid status transition is attempted (e.g., marking an order as "Ready" before "Preparing"), the system will reject the update and display an appropriate error message.	

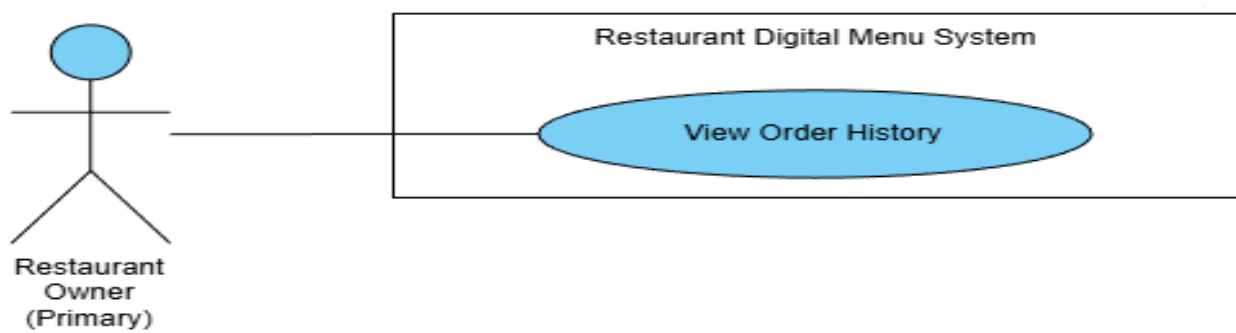


Figure 13 Use case View Order History

Table 24 Use case View Order History (UC_12)

Use Case ID	UC_12	
Use Case Name	View Order History	
Description	This use case allows the restaurant owner to access and review a list of all completed and past orders from the system's records.	
Primary Actor	Restaurant Owner	
Secondary Actor	None	
Pre-Condition	The owner is logged into the Management Interface (UC-08).	
Post-Condition	The owner is presented with a chronological list of previous orders, including details such as date, items, table number, and final status.	
Basic Flow	Actor Action	System Action
	- Owner navigates to the “Order History” or “Reports” section within the Management Interface. - Owner applies optional filters (e.g., by date range or order status). - Owner may select a specific order to view detailed itemization.	- System retrieves stored order data from the Database. - System applies the filters and displays the matching orders in a list, sorted chronologically with relevant details for each entry. - System expands the view to show the full order breakdown.
Alternate Flow	If no past orders exist for the selected filters, the system displays a message: “No order history available for the selected criteria.”	

Chapter 4: Design

4.1 Architecture Diagram

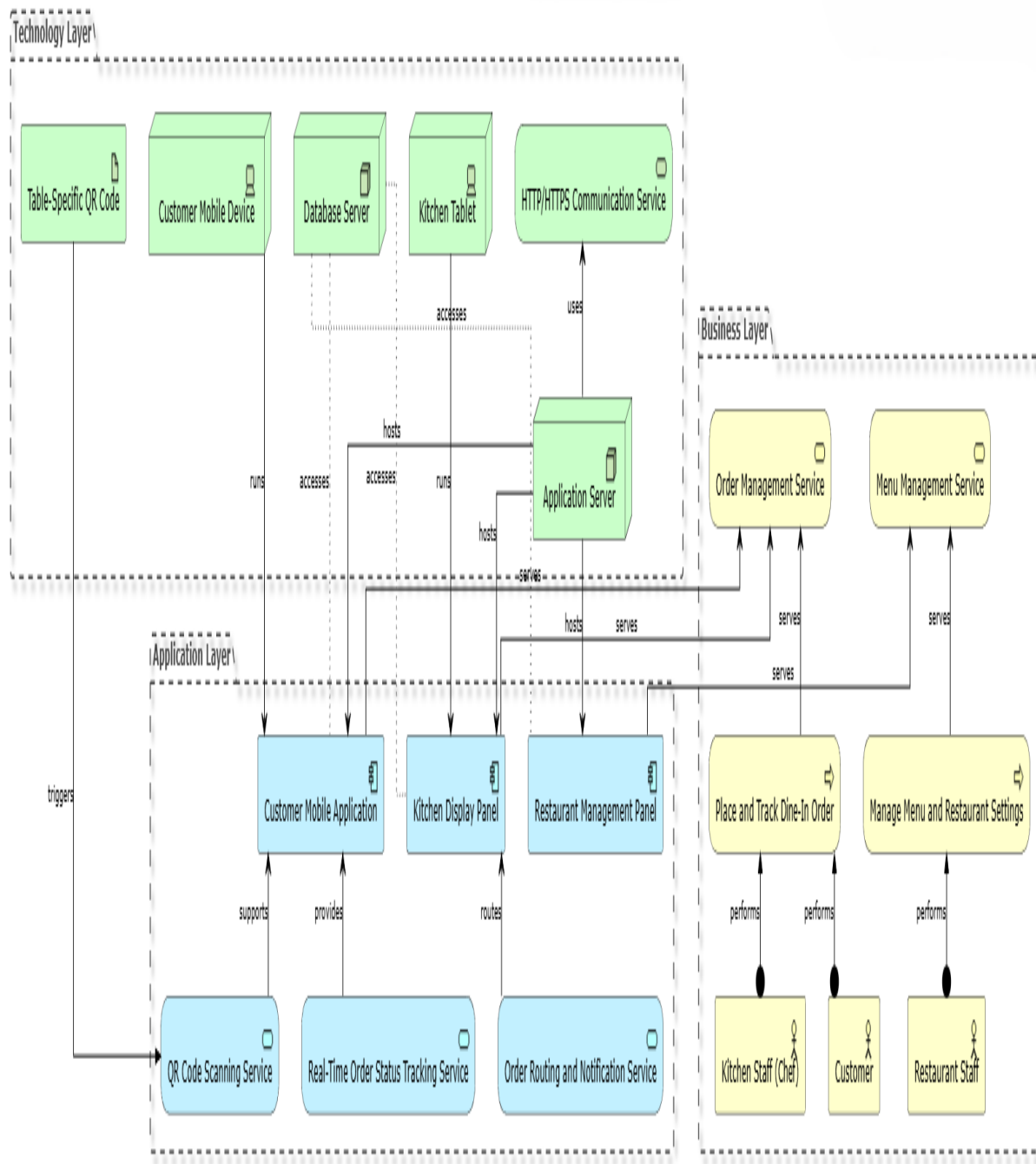


Figure 14 Architecture Diagram

4.2 ERD with data dictionary

Entity Relationship Diagram with complete relations with dependencies of your project

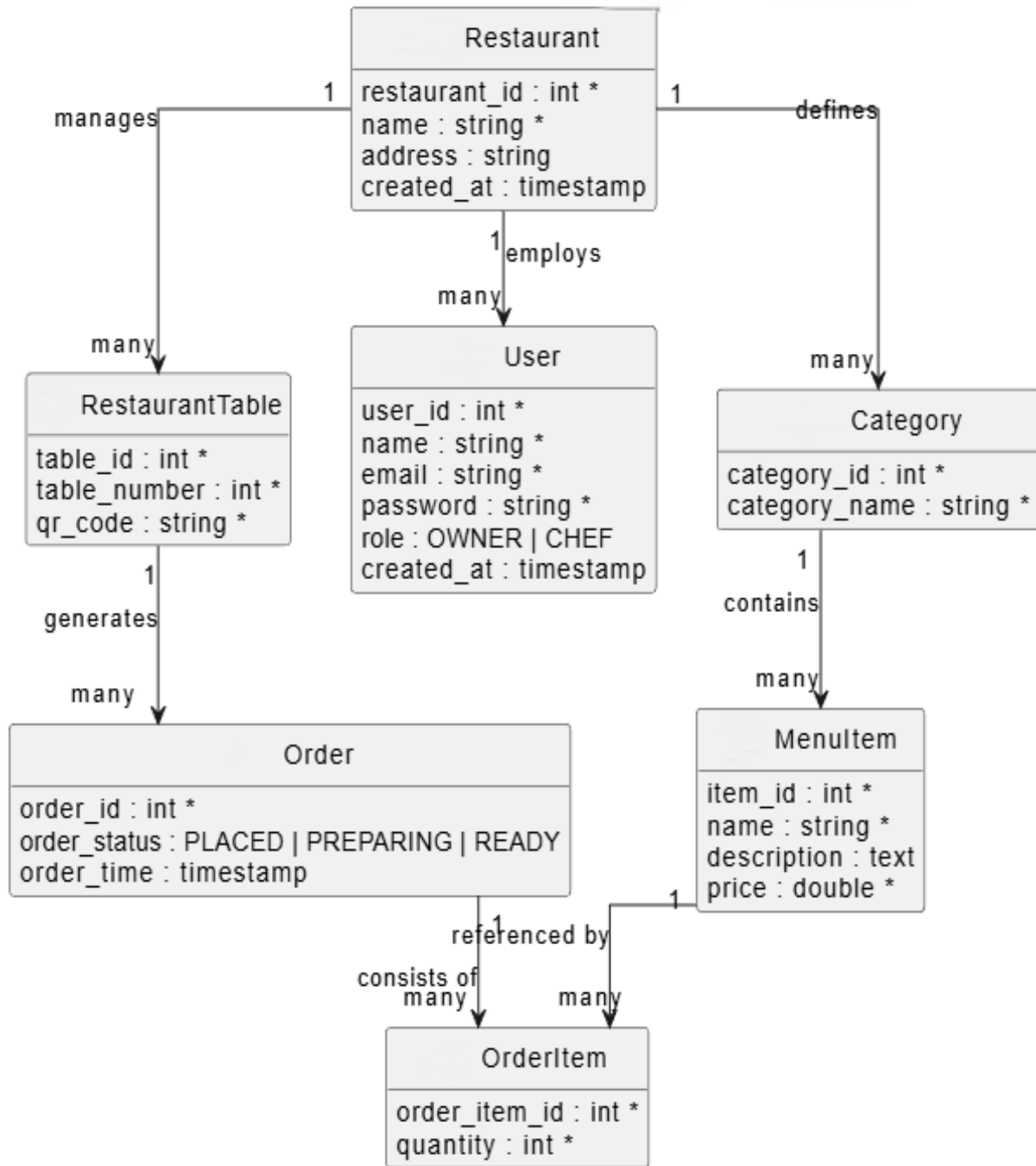


Figure 15 ERD

4.3 Data Flow Diagram

4.3.1 Level 0

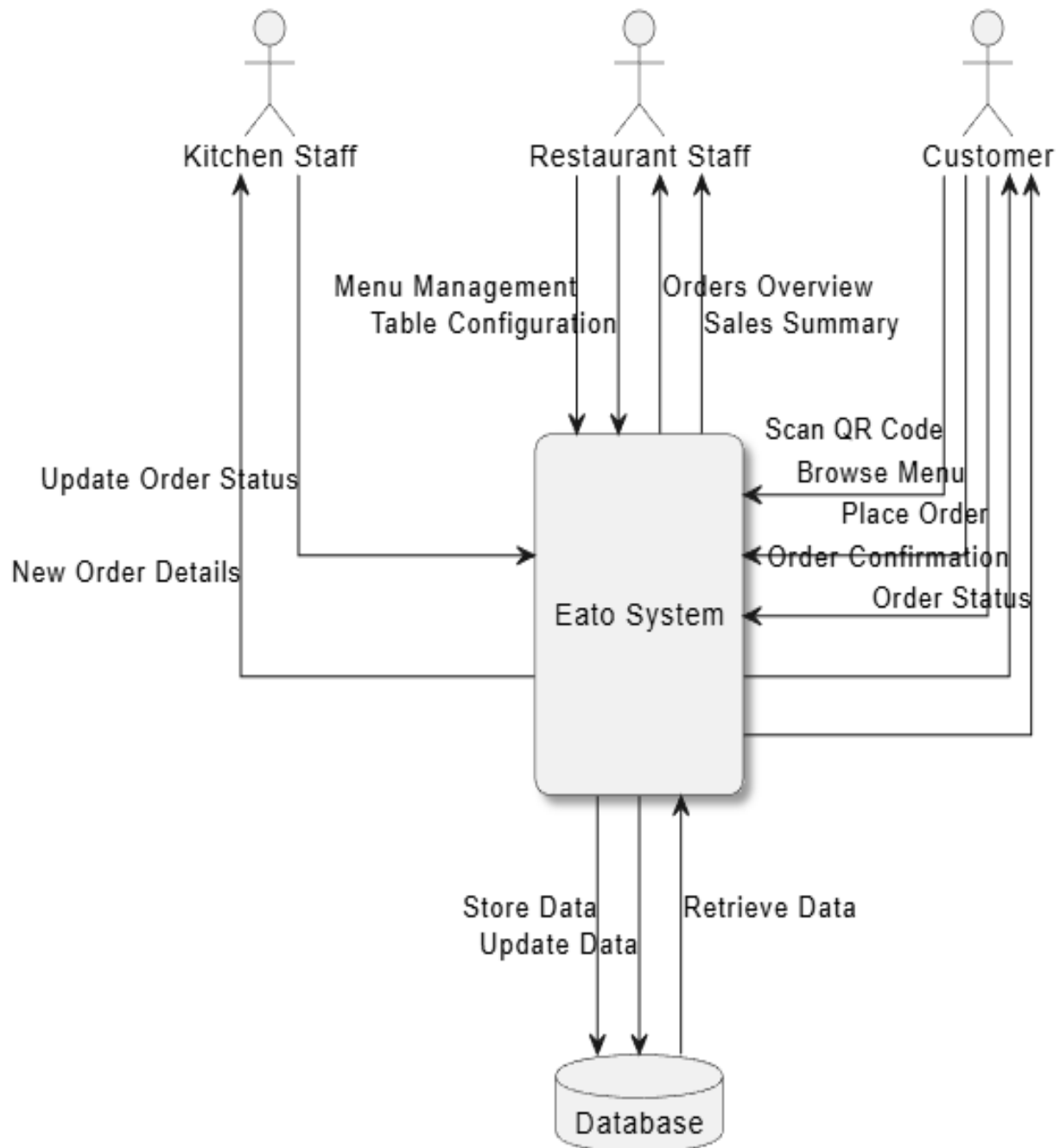


Figure 16 Level 0 DFD

4.3.2 Level 1

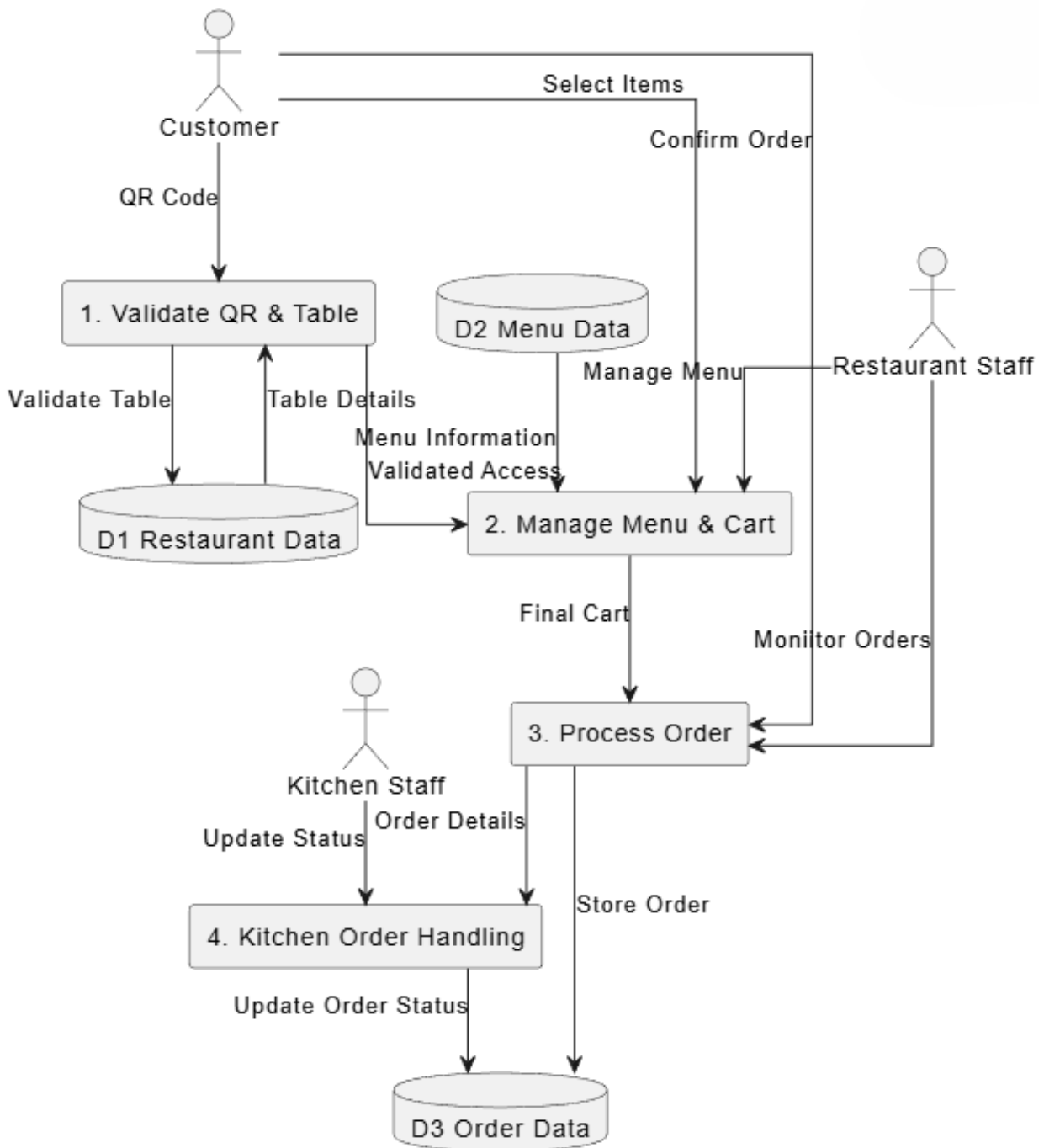


Figure 17 Level 1 DFD

4.4 Class Diagram

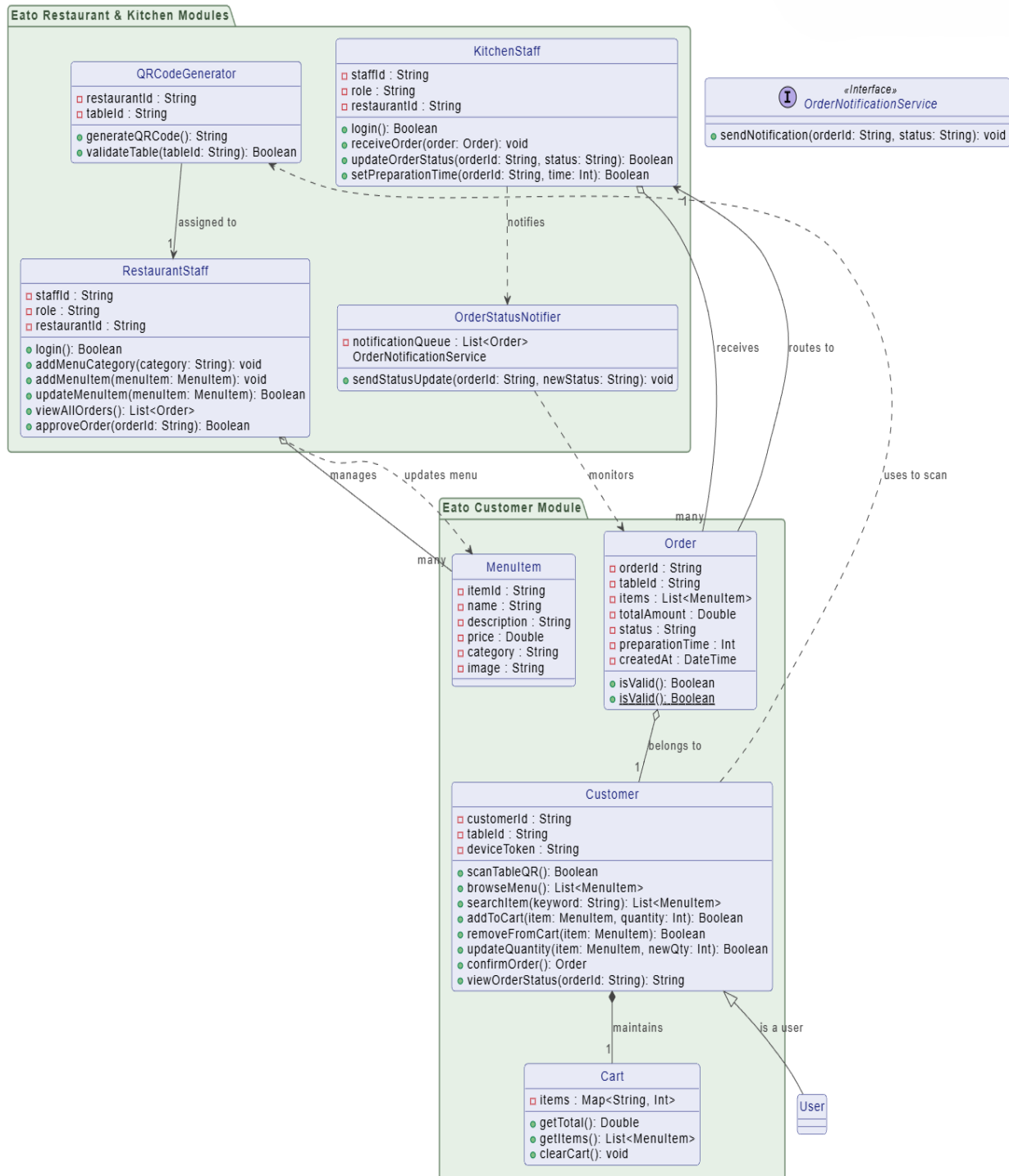


Figure 18 Class Diagram

4.5 Activity Diagram

4.5.1 Activity Diagram AD-01: Scan Table QR Code (FR-01)

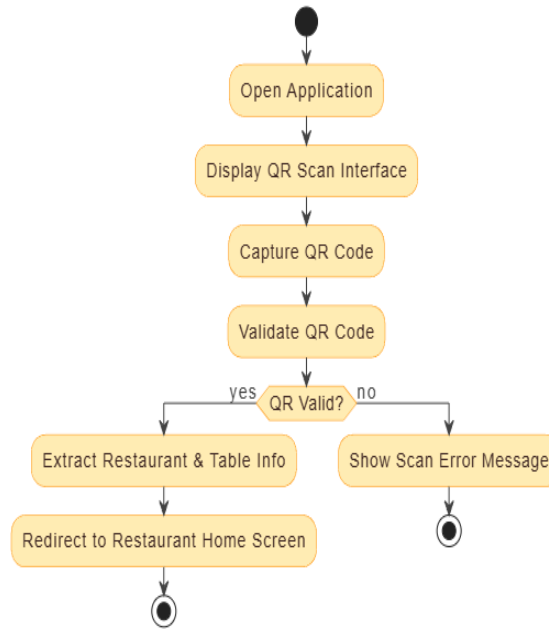


Figure 19 Activity Diagram Scan QR Code

4.5.2 Activity Diagram AD-02: View Digital Menu (FR-02)

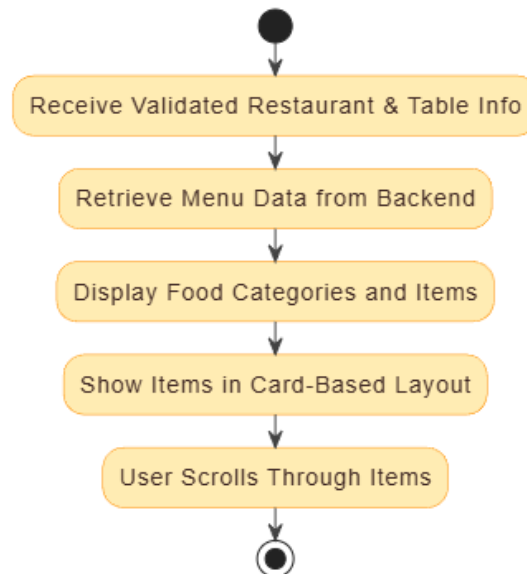


Figure 20 Activity Diagram View Digital Menu

4.5.3 Activity Diagram AD-03: Search for Food Items (FR-03)

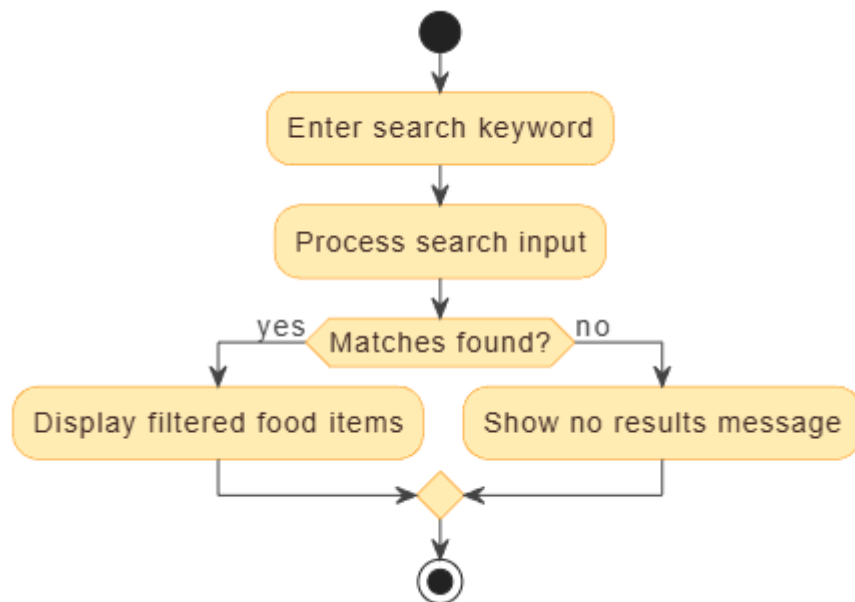


Figure 21 Activity Diagram Search for Food Items

4.5.4 Activity Diagram AD-04: Add Item to Cart (FR-04)



Figure 22 Activity Diagram Add Item to Cart

4.5.5 Activity Diagram AD-05: View and Manage Cart (FR-05)

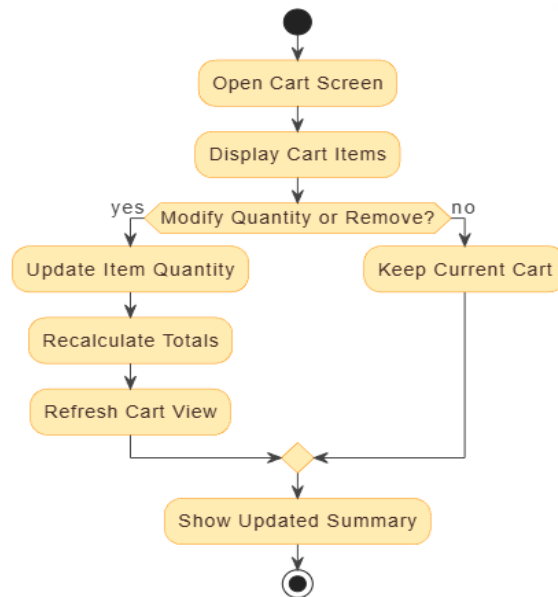


Figure 23 Activity Diagram View and Manage Cart

4.5.6 Activity Diagram AD-06: Place Order (FR-06)

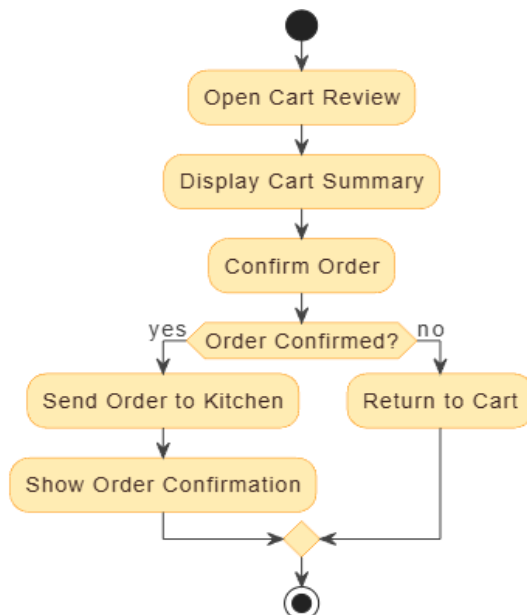


Figure 24 Activity Diagram Place Order

4.5.7 Activity Diagram AD-07: Track Order Status (FR-07)

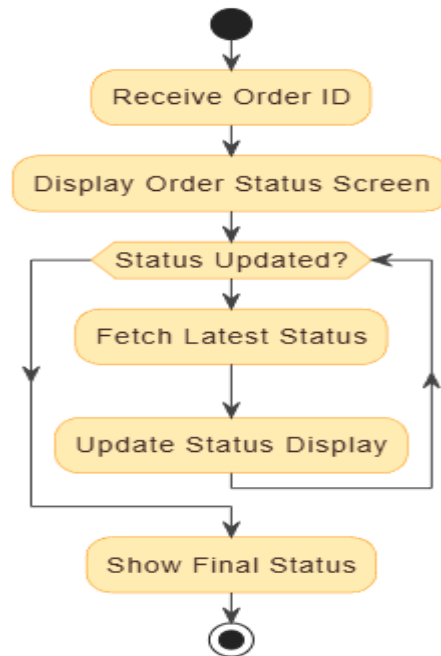


Figure 25 Activity Diagram Track Order Status

4.5.8 Activity Diagram AD-08: Owner Login (FR-08)

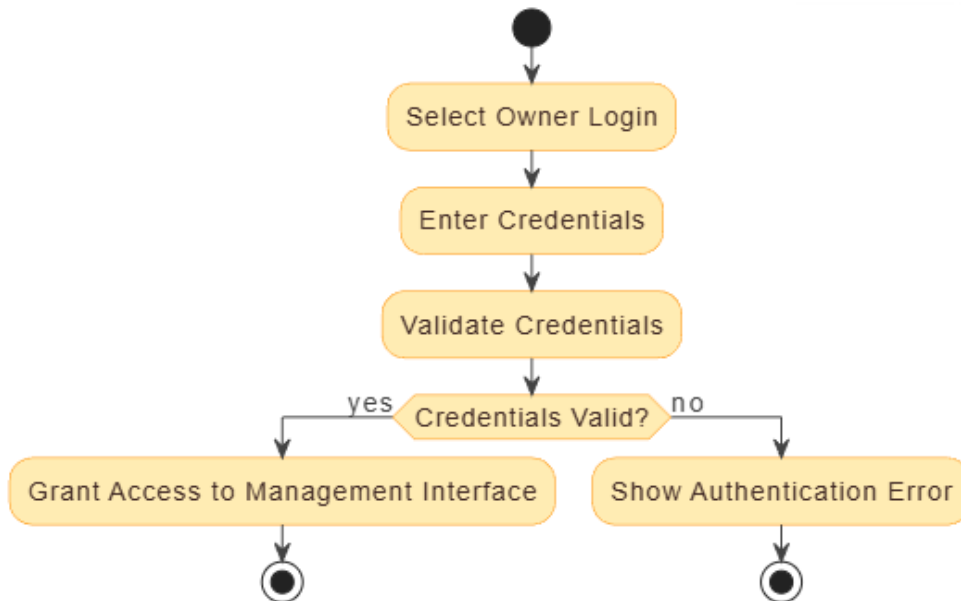


Figure 26 Activity Diagram Owner Login

4.5.9 Activity Diagram AD-09: Manage Menu Items (FR-09)

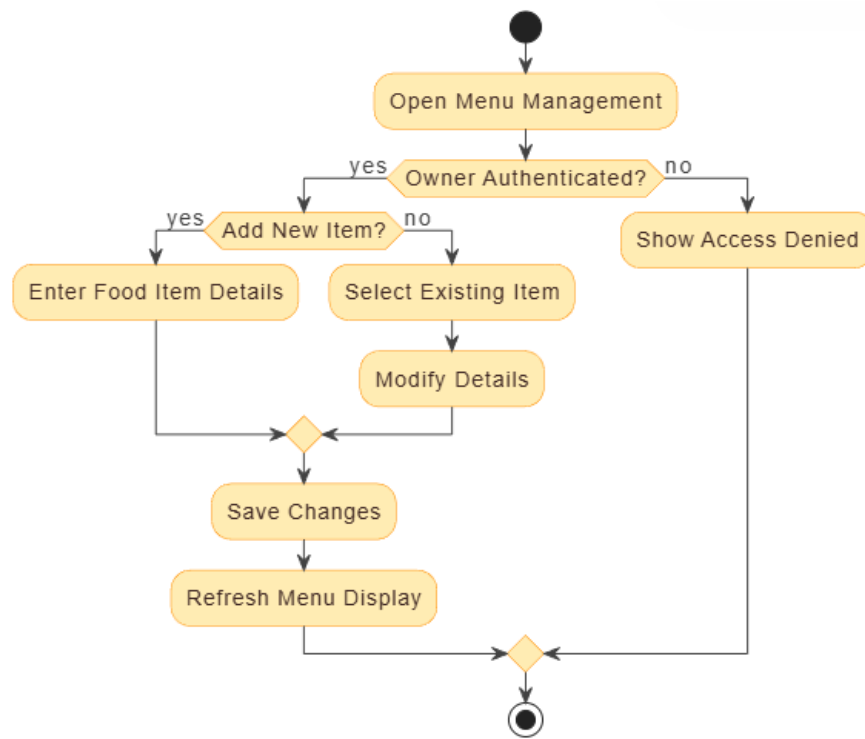


Figure 27 Activity Diagram Manage Menu Items

4.5.10 Activity Diagram AD-10: View Incoming Orders (FR-10)



Figure 28 Activity Diagram View Incoming Orders

4.6.11 Activity Diagram AD-11: Update Order Status (FR-11)

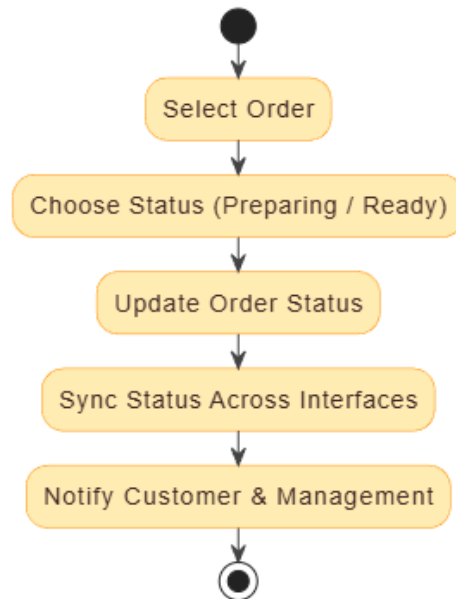


Figure 29 Activity Diagram Update Order Status

4.5.12 Activity Diagram AD-12: View Order History (FR-12)

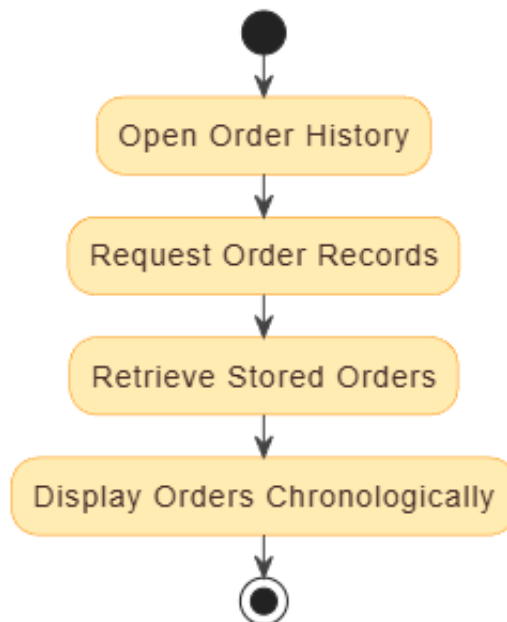


Figure 30 Activity Diagram View Order History

4.6 Sequence Diagram

4.6.1 Sequence Diagram SD-01: Scan Table QR Code (FR-01)

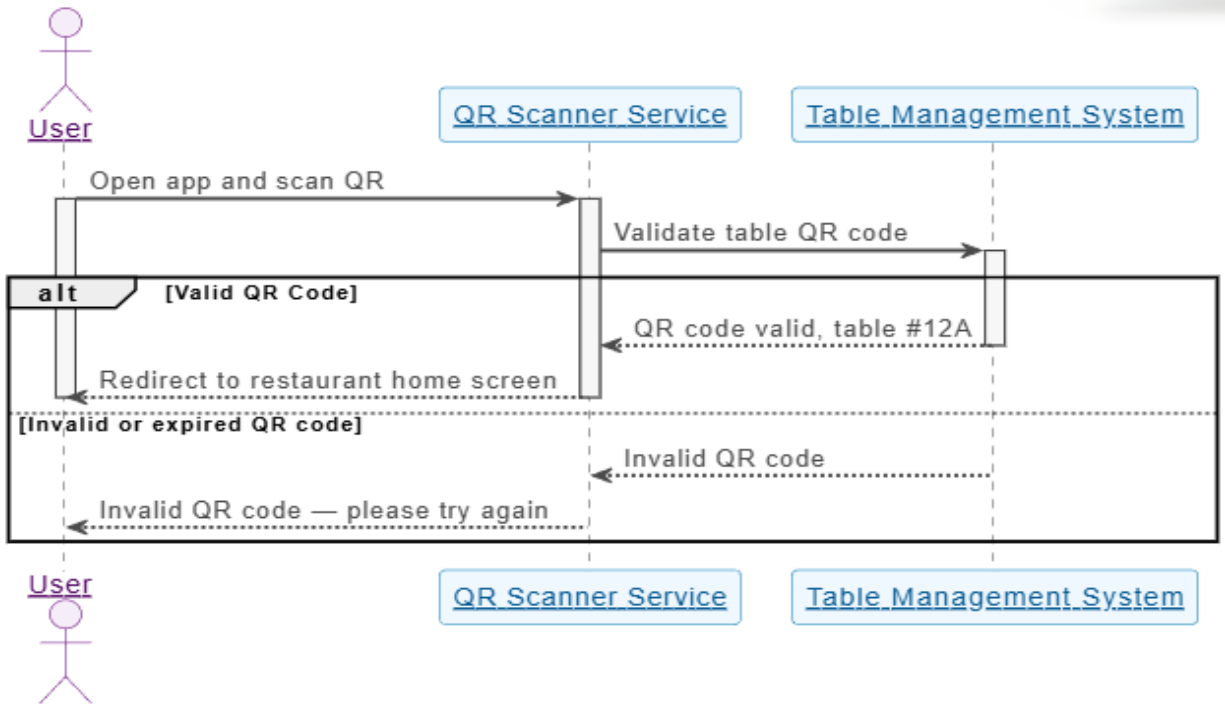


Figure 31 Sequence Diagram Scan QR Code

4.6.2 Sequence Diagram SD-02: View Digital Menu (FR-02)

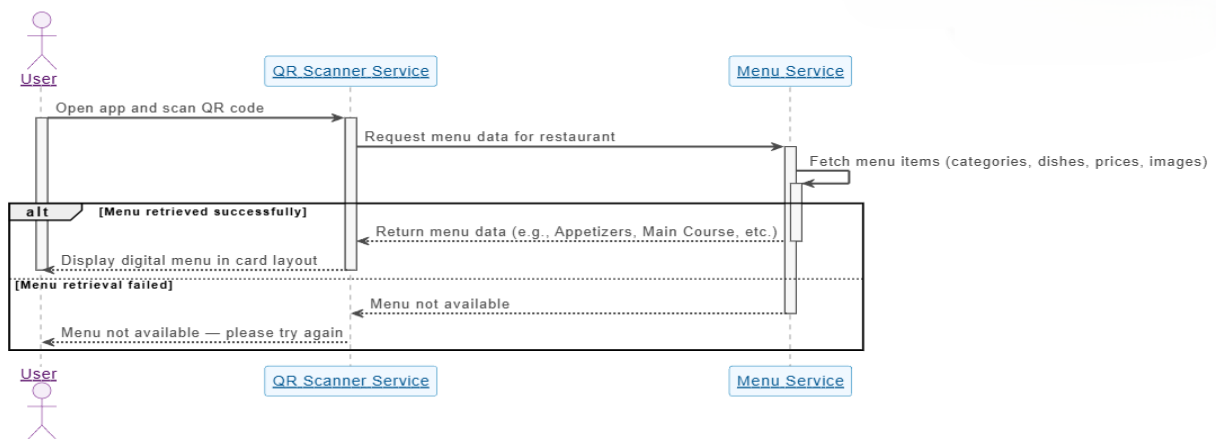


Figure 32 Sequence Diagram View Digital Menu

4.6.3 Sequence Diagram SD-03: Search Food Items (FR-03)

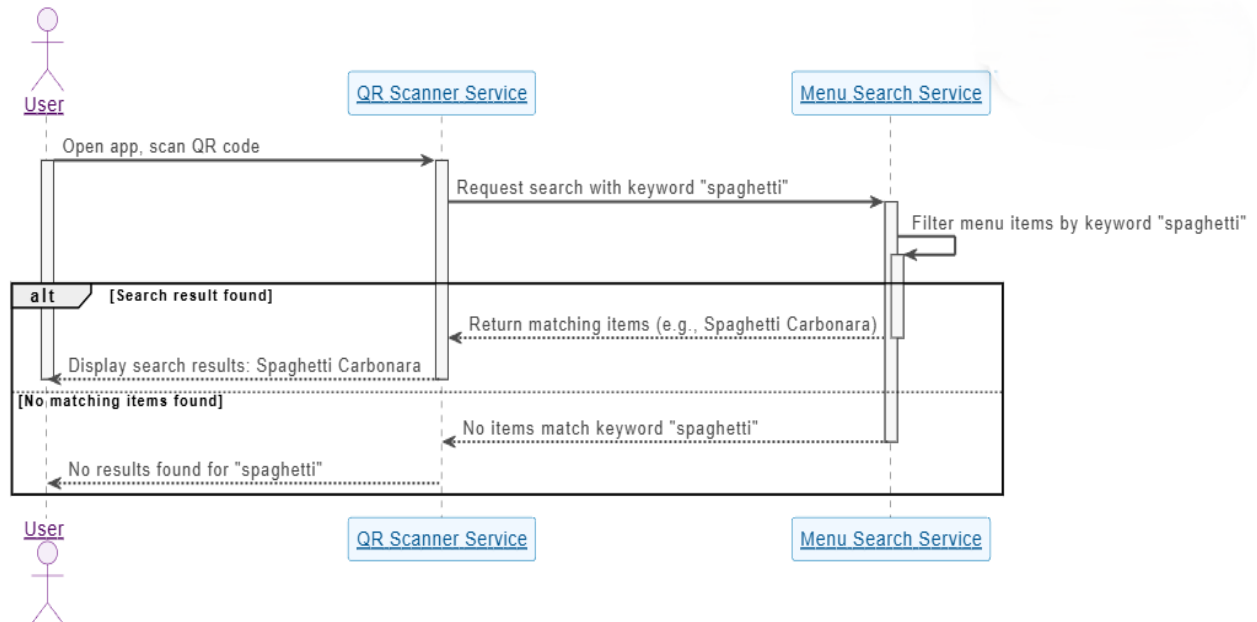


Figure 33 Sequence Diagram Search for Food Items

4.6.4 Sequence Diagram SD-04: Add Item to Cart (FR-04)

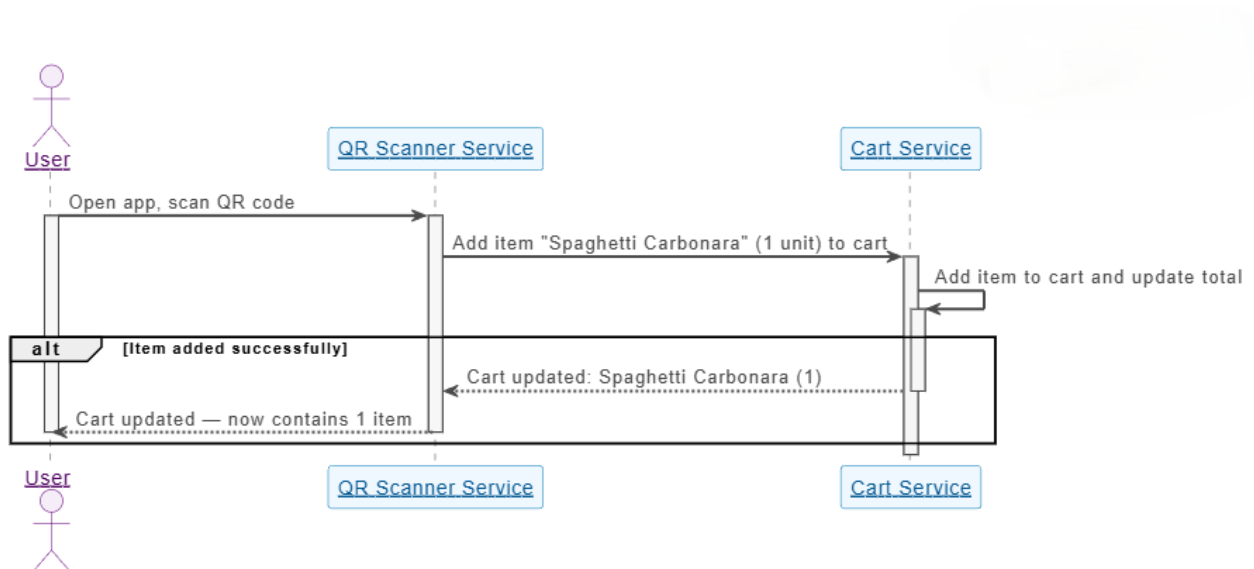


Figure 34 Sequence Diagram Add Item to Cart

4.6.5 Sequence Diagram SD-05: View and Manage Cart (FR-05)

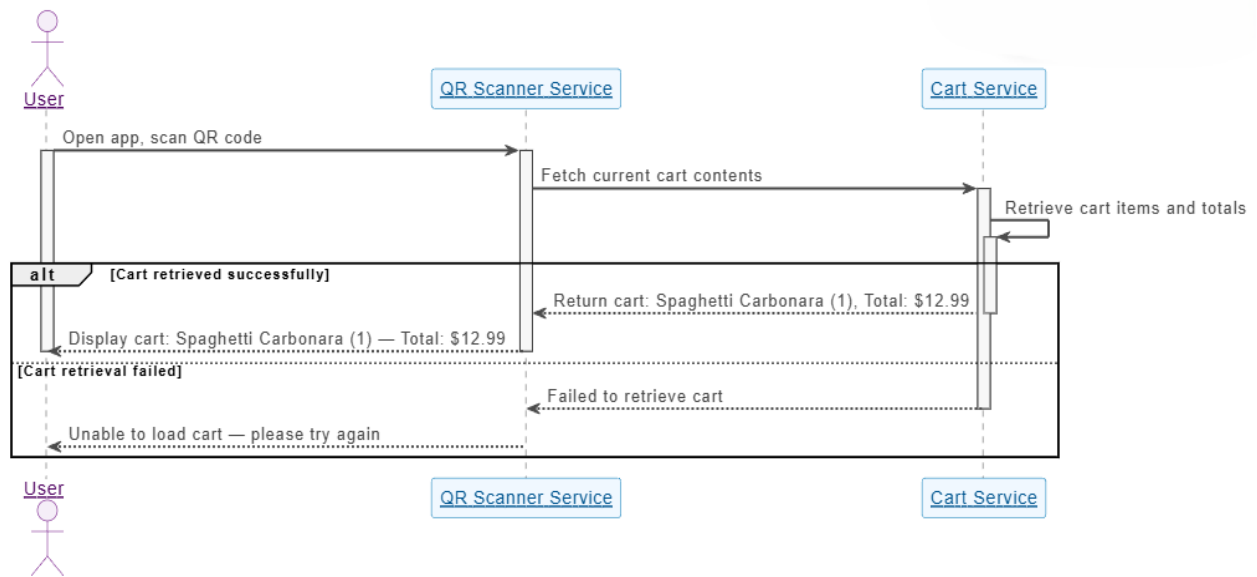


Figure 35 Sequence Diagram View and Manage Cart

4.6.6 Sequence Diagram SD-06: Place Order (FR-06)

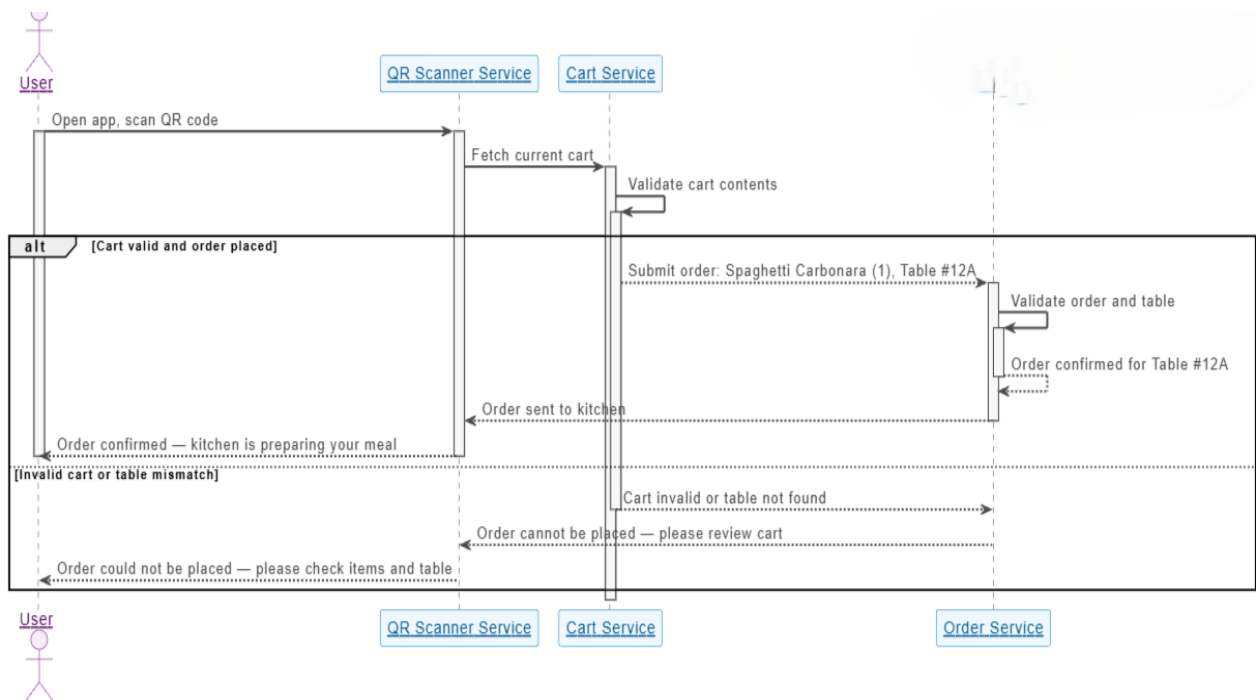


Figure 36 Sequence Diagram Place Order

4.6.7 Sequence Diagram SD-07: Track Order Status (FR-07)

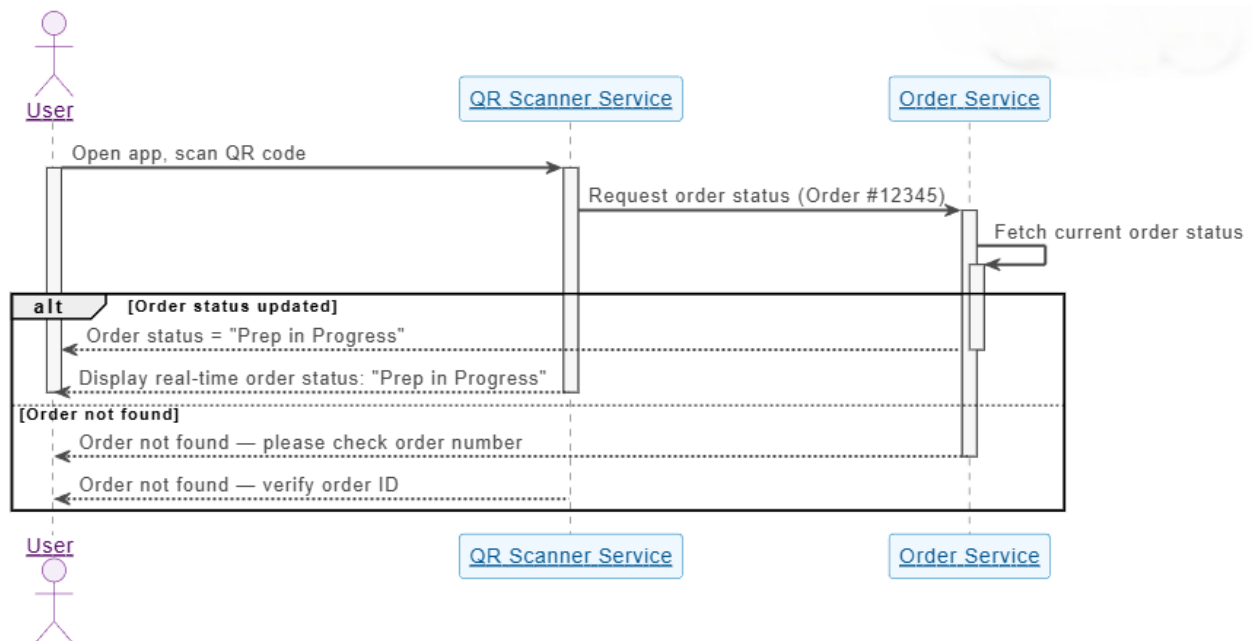


Figure 37 Sequence Diagram Track Order Status

4.6.8 Sequence Diagram SD-08: Owner Login (FR-08)

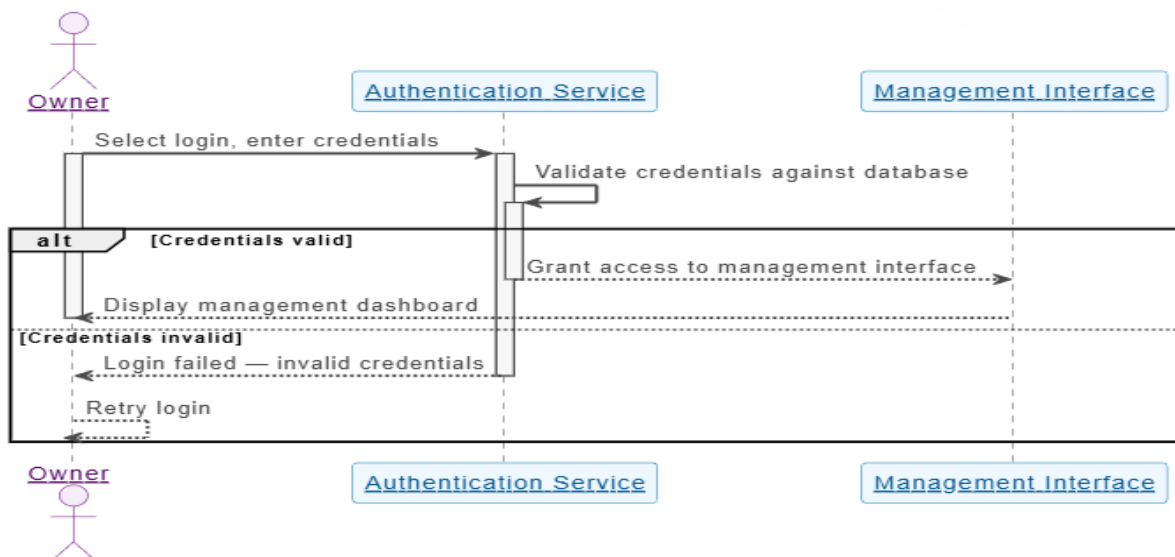


Figure 38 Sequence Diagram Owner Login

4.6.9 Sequence Diagram SD-09: Manage Menu Items (FR-09)

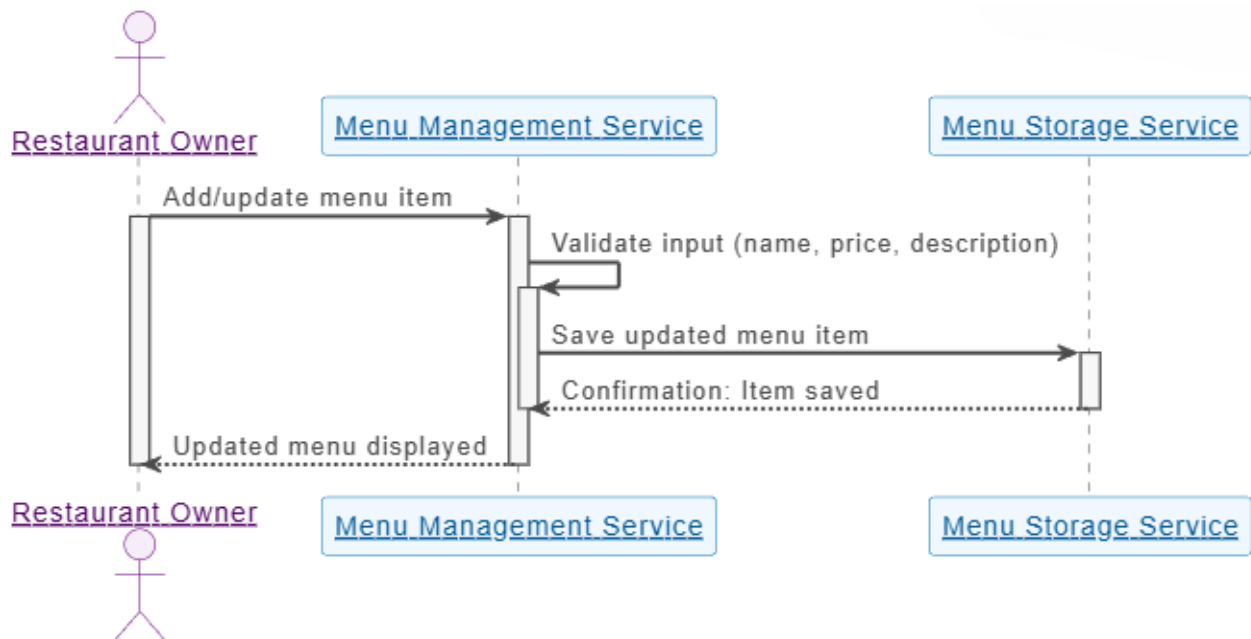


Figure 39 Sequence Diagram Manage Menu Items

4.6.10 Sequence Diagram SD-10: View Incoming Orders (FR-10)

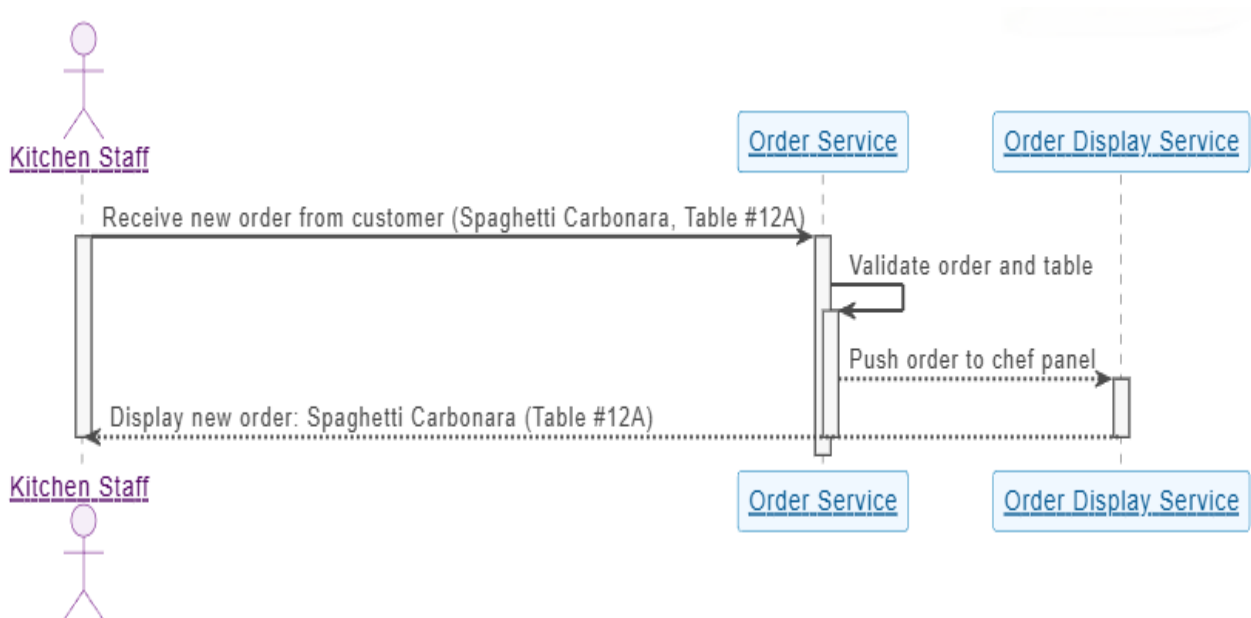


Figure 40 Sequence Diagram View Incoming Orders

4.6.11 Sequence Diagram SD-11: Update Order Status (FR-11)

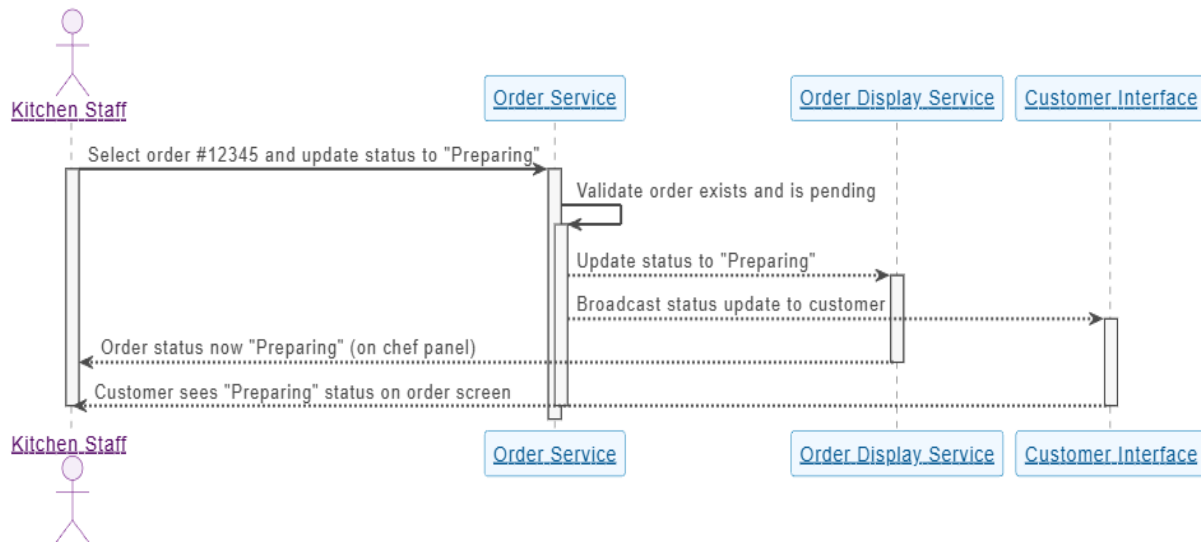


Figure 41 Sequence Diagram Update Order Status

4.6.12 Sequence Diagram SD-12: View Order History (FR-12)

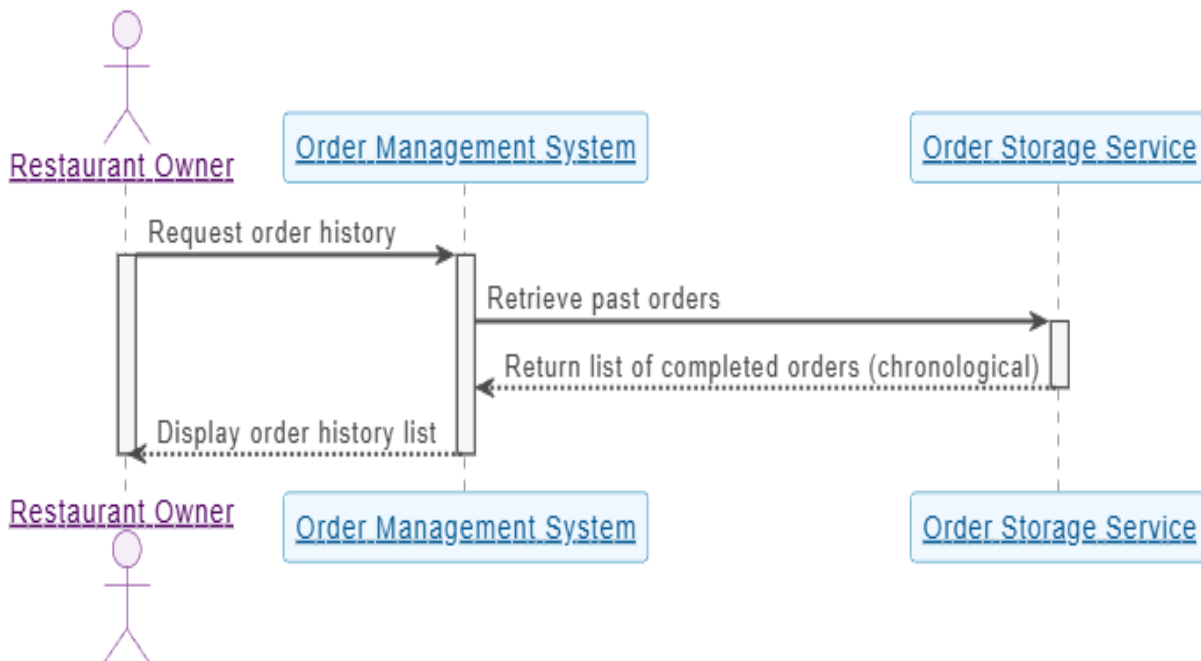


Figure 42 Sequence Diagram View Order History

4.7 Collaboration Diagram

4.7.1 Collaboration Diagram CD-01: Scan Table QR Code (FR-01)



Figure 43 Collaboration Diagram Scan Table QR Code

4.7.2 Collaboration Diagram CD-02: View Digital Menu (FR-02)



Figure 44 Collaboration Diagram View Digital Menu

4.7.3 Collaboration Diagram CD-03: Search Food Items (FR-03)



Figure 45 Collaboration Diagram Search Food Items

4.7.4 Collaboration Diagram CD-04: Add Item to Cart (FR-04)



Figure 46 Collaboration Diagram Add Item to Cart

4.7.5 Collaboration Diagram CD-05: View and Manage Cart (FR-05)

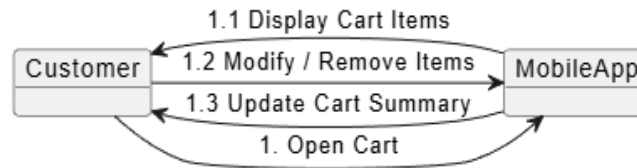


Figure 47 Collaboration Diagram View and Manage Cart

4.7.6 Collaboration Diagram CD-06: Place Order (FR-06)

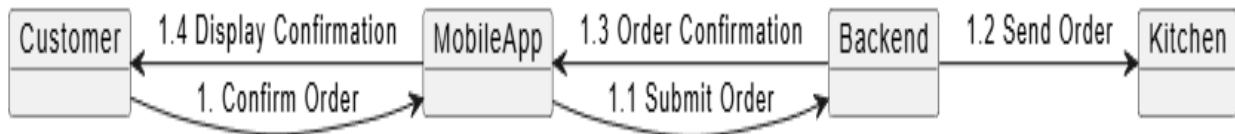


Figure 48 Collaboration Diagram Place Order

4.7.7 Collaboration Diagram CD-07: Track Order Status (FR-07)



Figure 49 Collaboration Diagram Track Order Status

4.7.8 Collaboration Diagram CD-08: Owner Login (FR-08)



Figure 50 Collaboration Diagram Owner Login

4.7.9 Collaboration Diagram CD-09: Manage Menu Items (FR-09)



Figure 51 Collaboration Diagram Manage Menu Items

4.7.10 Collaboration Diagram CD-10: View Incoming Orders (FR-10)

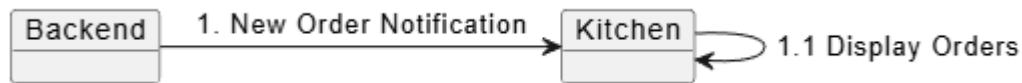


Figure 52 Collaboration Diagram View Incoming Orders

4.7.11 Collaboration Diagram CD-11: Update Order Status (FR-11)

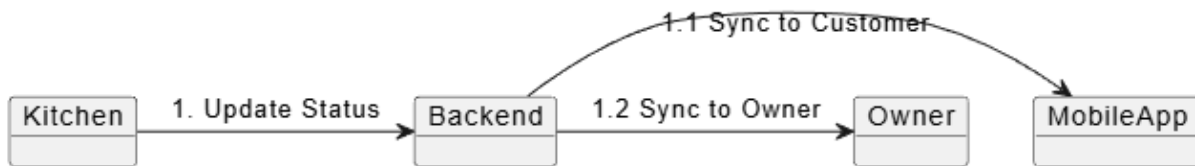


Figure 53 Collaboration Diagram Update Order Status

4.7.12 Collaboration Diagram CD-12: View Order History (FR-12)



Figure 54 Collaboration Diagram View Order History

4.8 State Transition Diagram

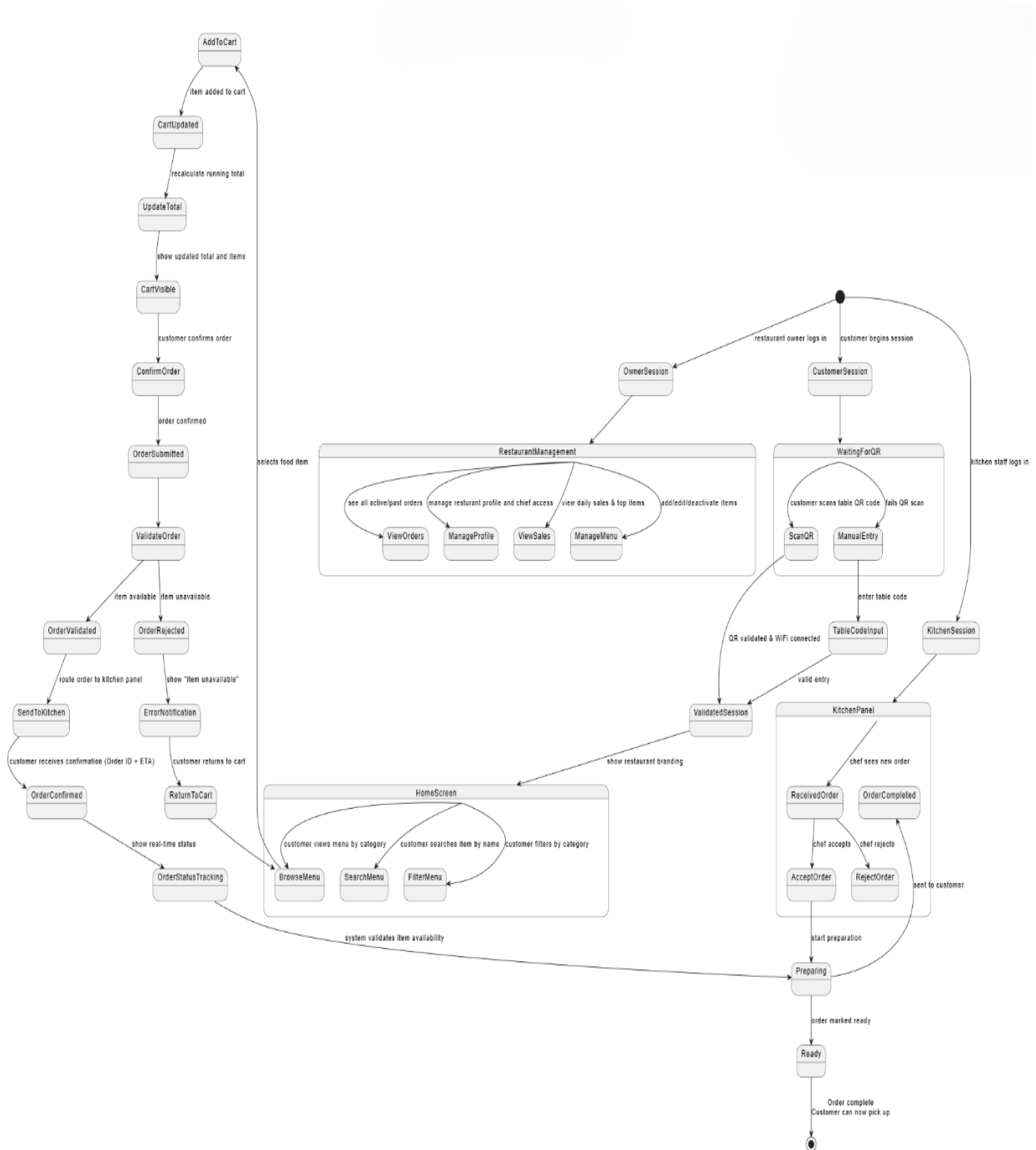


Figure 55 State Transition Diagram

4.9 Component Diagram

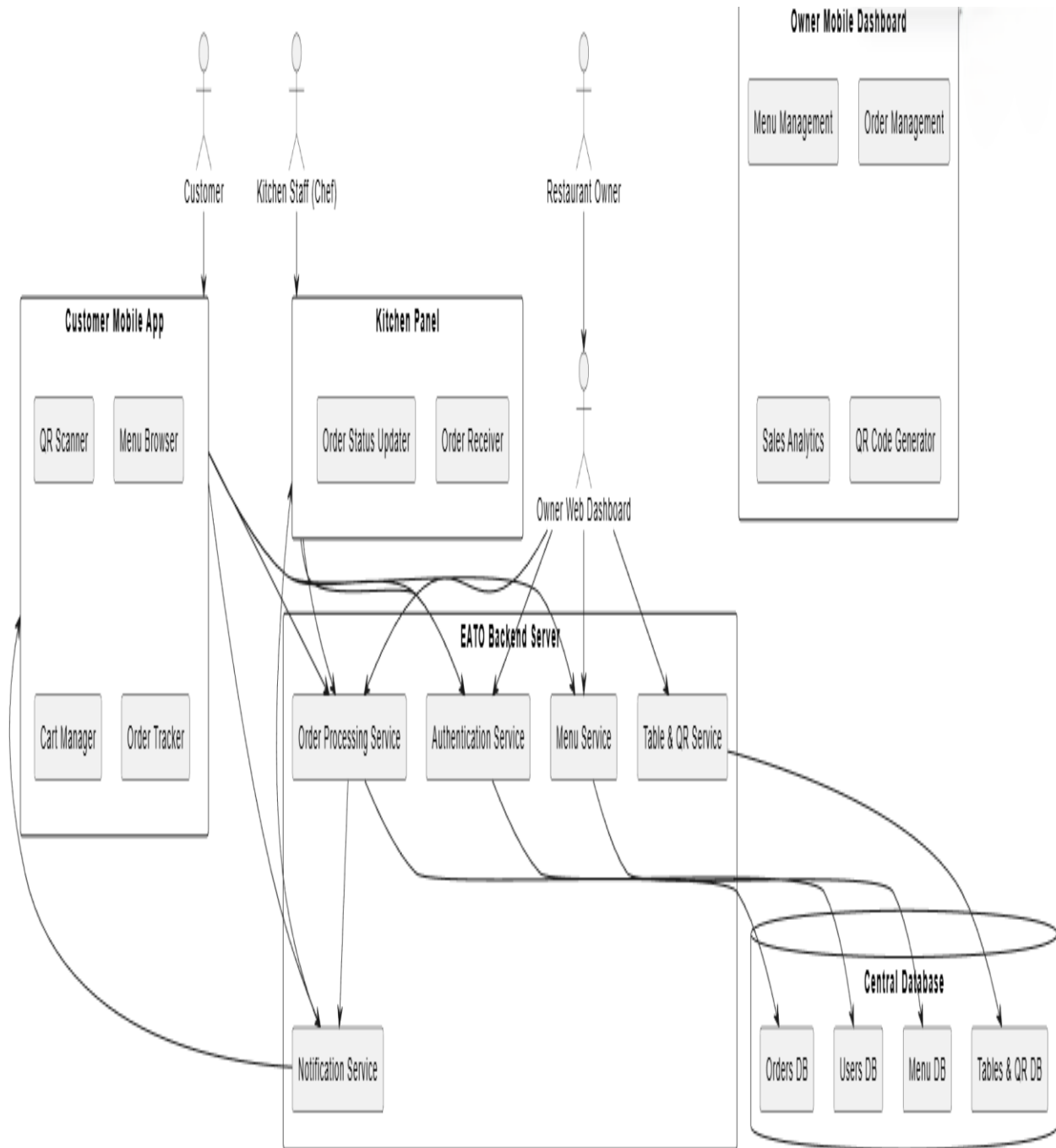


Figure 56 Component Diagram

4.10 Deployment Diagram

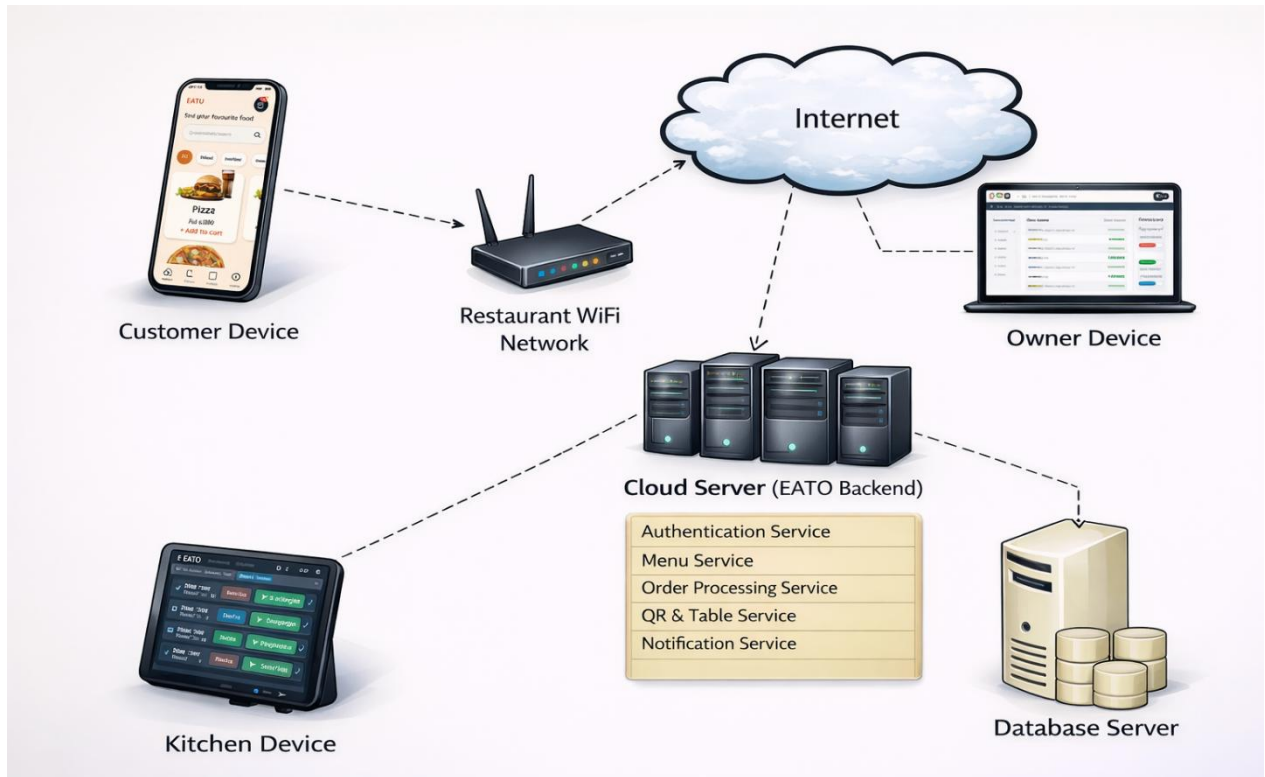


Figure 57 Deployment Diagram

Chapter 5: Testing

5.1 Test Case Specifications

5.1.1 Test Case for View Digital Menu

Table 25 Positive test case Menu View Success

Positive Test Case	
Field	Description
ID	TC_MENU_VIEW_SUCCESS
Priority	High
Description	To verify that the customer can successfully view the restaurant digital menu after QR validation.
Reference	FR-02 View Digital Menu
Users	Customer
Pre-requisites	Valid QR code has been scanned successfully.
Steps	A. Scan a valid QR code. B. Wait for the menu to load.
Input	Valid restaurant and table information
Expected Result	Food categories and menu items are displayed with images, descriptions, and prices.

Table 26 Negative Test Case Track Order Failure

Negative Test Case	
Field	Description
ID	TC_TRACK_ORDER_FAILURE
Priority	Medium
Description	To verify behavior when invalid order information is requested.
Reference	FR-07 Track Order Status
Users	Customer

Pre-requisites	None
Steps	A. Attempt to access invalid order information.
Input	Invalid Order ID
Expected Result	System displays an appropriate error message.
Status	Tested, Passed

5.1.2 Test Case for Search Food Items

Table 27 Positive Test Case Search Success

Positive Test Case	
Field	Description
ID	TC_SEARCH_SUCCESS
Priority	Medium
Description	To verify that customers can search for menu items using keywords.
Reference	FR-03 Search Food Items
Users	Customer
Pre-requisites	The digital menu is loaded successfully.
Steps	A. Open the search bar. B. Enter a food item keyword.
Input	Valid food item keyword
Expected Result	Matching food items are displayed in real-time.
Status	Tested, Passed

Table 28 Negative Test Case Search Failure

Negative Test Case	
Field	Description
ID	TC_SEARCH_FAILURE
Priority	Low

Description	To verify system behavior when no matching food item exists.
Reference	FR-03 Search Food Items
Users	Customer
Pre-requisites	Digital menu is loaded successfully.
Steps	A. Open the search bar. B. Enter an invalid keyword.
Input	Non-existing food item keyword
Expected Result	The system displays a “No Results Found” message.
Status	Tested, Passed

5.1.3 Test Case for Add Item to Cart

Table 29 Positive Test Case Cart Add Success

Positive Test Case	
Field	Description
ID	TC_CART_ADD_SUCCESS
Priority	High
Description	To verify that customers can successfully add menu items to the cart.
Reference	FR-04 Add Item to Cart
Users	Customer
Pre-requisites	Menu items are displayed.
Steps	A. Select a food item. B. Press Add to Cart.
Input	Valid food item selection
Expected Result	Selected items are added to the cart and the cart total updates automatically.
Status	Tested, Passed

Table 30 Negative Test Case Cart Add Failure

Negative Test Case	
Field	Description
ID	TC_CART_ADD_FAILURE
Priority	Medium
Description	To verify system behavior when an unavailable item is selected.
Reference	FR-04 Add Item to Cart
Users	Customer
Pre-requisites	Menu items displayed.
Steps	A. Attempt to add an unavailable item.
Input	Out-of-stock item
Expected Result	System prevents addition and displays an availability error message.
Status	Tested, Passed

5.1.4 Test Case for Place Order

Table 31 Positive Test Case Order Place success

Positive Test Case	
Field	Description
ID	TC_ORDER_PLACE_SUCCESS
Priority	High
Description	To verify successful order placement from customer cart.
Reference	FR-06 Place Order
Users	Customer
Pre-requisites	Cart contains at least one item.
Steps	A. Open cart. B. Press Place Order. C. Confirm order submission.

Input	Valid order details
Expected Result	Order is created successfully, stored in the database, and sent to Chef Panel in real-time.
Status	Tested, Passed

Table 32 Negative Test Case Order Place Failure

Negative Test Case	
Field	Description
ID	TC_ORDER_PLACE_FAILURE
Priority	High
Description	To verify system behavior when a customer attempts to place an empty order.
Reference	FR-06 Place Order
Users	Customer
Pre-requisites	Cart is empty.
Steps	A. Open empty cart. B. Press Place Order.
Input	Empty cart
Expected Result	System prevents order placement and displays a warning message.
Status	Tested, Passed

5.1.5 Test Case for Track Order Status

Table 33 Positive Test Case Track Order Success

Positive Test Case	
Field	Description
ID	TC_TRACK_ORDER_SUCCESS

Priority	High
Description	To verify real-time order status tracking functionality.
Reference	FR-07 Track Order Status
Users	Customer
Pre-requisites	Order has been placed successfully.
Steps	A. Open Track Order screen. B. Wait for kitchen status updates.
Input	Valid Order ID
Expected Result	System displays live order statuses such as Pending, Preparing, and Ready.
Status	Tested, Passed

Table 34 Negative Test Case Track order Success

Negative Test Case	
Field	Description
ID	TC_TRACK_ORDER_FAILURE
Priority	Medium
Description	To verify behavior when invalid order information is requested.
Reference	FR-07 Track Order Status
Users	Customer
Pre-requisites	None
Steps	A. Attempt to access invalid order information.
Input	Invalid Order ID
Expected Result	System displays an appropriate error message.
Status	Tested, Passed

5.1.6 Test Case for Owner Login

Table 35 Positive Test Case Owner Login Success

Positive Test Case	
Field	Description
ID	TC_OWNER_LOGIN_SUCCESS
Priority	High
Description	To verify successful owner authentication.
Reference	FR-08 Owner Login
Users	Owner/Admin
Pre-requisites	Owner accounts exist in the system.
Steps	A. Open the owner login page. B. Enter email and password. C. Press Login.
Input	Valid owner credentials
Expected Result	Owner is authenticated and redirected to the management dashboard.
Status	Tested, Passed

Table 36 Negative Test Case Owner Login Failure

Negative Test Case	
Field	Description
ID	TC_OWNER_LOGIN_FAILURE
Priority	High
Description	To verify behavior when invalid credentials are entered.
Reference	FR-08 Owner Login
Users	Owner/Admin
Pre-requisites	Login page accessible.

Steps	A. Enter incorrect email or password. B. Press Login.
Input	Invalid credentials
Expected Result	System denies access and displays an authentication error message.
Status	Tested, Passed

5.1.7 Test Case for Update Order Status

Table 37 Positive Test Case Status update Success

Positive Test Case	
Field	Description
ID	TC_STATUS_UPDATE_SUCCESS
Priority	High
Description	To verify that kitchen staff can update order statuses successfully.
Reference	FR-11 Update Order Status
Users	Chef/Kitchen Staff
Pre-requisites	Incoming order available in Chef Panel.
Steps	A. Select the incoming order. B. Update status to Preparing or Ready.
Input	Valid order status
Expected Result	Updated status is synchronized in real-time across customer and owner interfaces.
Status	Tested, Passed

Table 38 Negative Test Case Status Update Failure

Negative Test Case	
Field	Description
ID	TC_STATUS_UPDATE_FAILURE
Priority	Medium
Description	To verify system behavior for invalid order status transitions.
Reference	FR-11 Update Order Status
Users	Chef/Kitchen Staff
Pre-requisites	Order exists in Chef Panel.
Steps	A. Attempt invalid status transition.
Input	Invalid status update
Expected Result	System rejects invalid transition and displays appropriate error message.
Status	Tested, Passed

5.2 Black Box Test Cases

Black box testing, also known as Behavioral Testing, is a software testing method in which the internal structure, design, or implementation of the application is not visible to the tester [14]. This testing technique was used in Eato to validate customer ordering, authentication, QR scanning, and real-time order management functionalities.

The following errors were identified during black box testing:

- Incorrect or missing functions
- Interface errors
- Errors in database communication
- Behavior or performance errors
- Initialization and termination errors

5.2.1 Equivalence Partitions (EP)

Table 39 Equivalence Partitions

Variables	Valid Classes	Invalid Classes
Owner Email	Valid registered email format	Invalid email format or empty field
Password	Password length greater than 5 characters	Password length less than 5 characters
QR Token	Valid generated QR token	Invalid or expired QR token
Order Status	Pending, Preparing, Ready	Unknown or unsupported status

5.2.2 Boundary Value Analysis

Boundary Value Analysis was performed on password length validation, menu item quantities, and search input fields to ensure that minimum and maximum acceptable values were handled correctly.

5.2.3 Decision Table Testing

Decision table testing was used to verify conditions such as owner authentication, order submission, and chef order status updates.

5.2.4 State Transition Testing

State transition testing was used to verify correct movement between order states in the Eato system.

Table 40 State Transition testing

Current State	Action	Next State
Active	Customer Place order	Pending
Pending	Chef accepts order	Preparing
Preparing	Chef completes order	Ready

5.2.5 Use Case Testing

Use case testing was performed to validate complete user workflows such as scanning QR codes, placing orders, managing menus, and updating order statuses.

5.3 White Box Test Cases

White box testing was performed to validate internal code logic, route handling, middleware execution, database queries, and Socket.io event communication.

5.3.1 Cyclomatic Complexity

Cyclomatic complexity analysis was applied on authentication routes, order management logic, and status update modules to reduce logical complexity and improve maintainability.

5.4 Performance Testing

Performance testing was performed to evaluate application responsiveness, real-time synchronization, and API response times under multiple customer requests.

5.5 Stress Testing

Stress testing was performed by generating multiple simultaneous orders and real-time updates to verify that the backend server and database remained stable under heavy load.

5.6 System Testing

System testing was conducted on the complete Eato application, including customer interface, owner dashboard, chef panel, QR validation system, database operations, and real-time communication.

5.7 Regression Testing

Regression testing was carried out after implementing new features and fixing bugs to ensure that existing functionalities continued to work correctly.

5.7.1 Selecting Regression Tests

- Authentication modules
- QR scanning workflow
- Order placement functionality
- Socket.io real-time synchronization
- Chef order status updates
- Database CRUD operations

5.7.2 Regression Testing Steps

- Select critical test cases for regression.
- Execute automated and manual regression tests.
- Verify API responses and database consistency.
- Validate real-time order synchronization.
- Analyze and document results.

Chapter 6: Tools and Techniques

6.1 Introduction

This chapter presents the tools, technologies, programming languages, frameworks, libraries, and development environments used in the implementation of the Eato system. The selection of these technologies was based on their reliability, scalability, performance, and compatibility with modern mobile application development practices. The Eato system follows a full-stack development approach consisting of a mobile frontend, a backend server, a cloud-hosted database, and real-time communication services [15].

6.2 Programming Languages

JavaScript was the primary programming language used throughout the project. It was utilized for frontend development, backend development, API creation, business logic implementation, and real-time communication functionalities [16].

TypeScript was used within the React Native frontend to provide static type checking and improve code reliability. It helped reduce runtime errors and enhanced the maintainability of the application [17].

Although React Native does not directly use HTML and CSS, similar styling principles were applied through NativeWind and Tailwind CSS utilities to create responsive and visually appealing user interfaces [18].

6.3 Development Tools and Applications

- Visual Studio Code was used as the primary source code editor for both frontend and backend development [17].
- Android Studio was used for Android device emulation, testing, debugging, and application deployment [18].
- MongoDB Atlas was used as the cloud-hosted NoSQL database [19].
- Render was used as the cloud hosting platform for deploying the backend server [20].
- Expo was used to simplify React Native application development and testing [21].
- Cloudinary was used for image storage and media management [22].

6.4 Frameworks and Libraries

- React Native was used to develop the mobile application interface [16].
- Express.js was used for backend development and API creation [23].
- Node.js served as the runtime environment for executing backend JavaScript code.
- Mongoose was used as an Object Data Modeling library for MongoDB.
- Socket.io was implemented to enable real-time communication between customers, chefs, and restaurant owners.
- JWT was used for authentication and authorization [12].
- Bcrypt.js was used for password hashing.
- Axios was used for communication between the frontend and backend APIs [24].

6.5 State Management Libraries

Redux Toolkit was used to manage global application state and simplify state management across multiple screens [25].

6.6 User Interface and Styling Libraries

Native Wind was used to integrate Tailwind CSS styling principles into React Native development. Tailwind CSS utility classes were used through Native Wind to create responsive and modern user interfaces.

6.7 Additional Libraries and Packages

- react-native-qrcode-svg for QR code generation [8].
- expo-camera for QR code scanning and camera access.
- expo-image-picker for image selection.
- react-navigation for screen navigation.
- react-native-svg for SVG rendering support.
- multer for file uploads.
- helmet for API security.
- cors for cross-origin resource sharing.
- cookie-parser for cookie management.
- morgan for request logging.

6.8 Summary

The Eato system was developed using modern full-stack technologies including React Native, Expo, Node.js, Express.js, MongoDB Atlas, Socket.io, and Cloudinary. These technologies provided a scalable, secure, and efficient platform for implementing a QR-based restaurant ordering and management system. The selected technologies enabled real-time communication, cloud-based storage, secure authentication, and a responsive user experience, contributing significantly to the overall success of the project.

Chapter 7: Summary and Conclusion

7.1 Summary

The restaurant industry continuously seeks innovative solutions to improve customer experience, operational efficiency, and service quality. Traditional restaurant ordering methods often involve manual order taking, communication delays between customers and kitchen staff, and increased chances of human error. To address these challenges, the Eato system was developed as a QR-based restaurant ordering and management platform.

The primary objective of Eato is to provide a digital ordering environment where customers can scan a QR code placed on a restaurant table, browse the menu, place orders, and track order progress without requiring direct interaction with restaurant staff. The system streamlines restaurant operations by connecting customers, restaurant owners, and kitchen staff through a unified platform.

The application was developed using modern full-stack technologies including React Native, Expo, Node.js, Express.js, MongoDB Atlas, Socket.io, Cloudinary, and JWT authentication. The use of these technologies enabled the implementation of a scalable, secure, and responsive mobile application capable of supporting real-time order processing and synchronization.

The Eato system consists of three primary user roles:

- Customer
- Owner/Admin
- Chef/Kitchen Staff

Customers can scan table QR codes, view restaurant menus, search food items, manage their carts, place orders, and track order status in real time. Restaurant owners can manage menus, monitor orders, view restaurant activities, and maintain restaurant information. Kitchen staff can receive incoming orders and update order statuses, allowing customers and owners to receive live updates through Socket.io based communication.

Throughout the development process, software engineering principles were applied during requirement gathering, system analysis, design, implementation, and testing. Multiple testing

techniques including black box testing, white box testing, system testing, stress testing, and regression testing were conducted to ensure system reliability and performance.

The results demonstrate that the Eato system successfully fulfils its intended objectives and provides an effective digital solution for restaurant order management.

7.2 Achievement of Objectives

The objectives defined at the beginning of the project were successfully achieved.

- The first objective was to develop a QR-based ordering system that eliminates the need for traditional paper menus and manual order-taking processes. This objective was successfully implemented through QR code generation and scanning functionality.
- The second objective was to improve communication between customers and kitchen staff through real-time order updates. This was achieved using Socket.io, which enabled instant synchronization of order statuses across the application.
- The third objective was to provide restaurant owners with a centralized platform for managing menus, orders, and restaurant information. The owner management interface successfully supports these functionalities.
- The fourth objective was to create an intuitive and user-friendly mobile application. Through responsive design and streamlined workflows, customers can place orders efficiently with minimal effort.

The successful implementation of these objectives confirms that the proposed solution effectively addresses the identified problems in traditional restaurant ordering systems.

7.3 Benefits of the Proposed System

The Eato system provides several benefits to both restaurant operators and customers.

7.3.1 Benefits for Customers

- Faster and more convenient ordering process.
- Reduced waiting times.
- Real-time order tracking.
- Easy menu browsing and searching.
- Contactless ordering experience.

7.3.2 Benefits for Restaurant Owners

- Improved order management.
- Reduced human errors.
- Better operational efficiency.
- Centralized menu administration.
- Enhanced customer satisfaction.

7.3.3 Benefits for Kitchen Staff

- Instant order notifications.
- Organized order workflow.
- Efficient status management.
- Improved communication with customers and management.

7.4 Project Limitations

Although the Eato system successfully achieves its primary objectives, several limitations currently exist.

- The system requires an active internet connection for proper functionality.
- Online payment integration has not yet been implemented.
- Push notification services are currently limited.
- The application is primarily designed for restaurant environments and may require customization for other food service businesses.
- Advanced business analytics and reporting features are not currently available.
- These limitations provide opportunities for future enhancement and expansion.

7.5 Conclusion

The Eato system successfully demonstrates the application of modern mobile and web technologies in solving real-world restaurant management challenges. By combining QR-based ordering, real-time communication, cloud-based storage, and role-based management interfaces, the system provides a complete digital ordering solution for restaurants.

The project achieved its primary goals of simplifying customer ordering, improving communication between customers and kitchen staff, and enhancing restaurant operational

efficiency. The implementation of real-time order tracking and centralized restaurant management contributes to a more organized and effective restaurant workflow.

The successful development and testing of Eato confirms the feasibility of adopting digital restaurant management systems to replace traditional manual processes. The system provides a strong foundation for future enhancements and demonstrates the potential of technology-driven solutions within the restaurant industry.

Overall, Eato represents a practical, scalable, and efficient restaurant ordering platform capable of improving service quality, customer satisfaction, and operational productivity.

Chapter 8 User Manual

This chapter provides a step-by-step guide on how to use the Eato application for both customers and restaurant owners. Screenshots are included to illustrate the user interface and workflow of the application.

8.1 Customer Application

8.1.1 Launching the App and Scanning QR Code

The customer app opens the camera automatically to scan the QR code placed on the restaurant table. The QR code connects the user to the restaurant and table.

Steps:

1. Open the Eato mobile app.
2. Point the device camera at the QR code on the restaurant table.
3. Wait for the QR code to be scanned and verified.
4. After scanning, the restaurant home screen will load, displaying restaurant branding and menu categories.
5. You can easily select the items from the given menu and add them to the cart.
6. If there is an error while scanning the QR code, the app will through an alert message. Just press the ok button and try again.

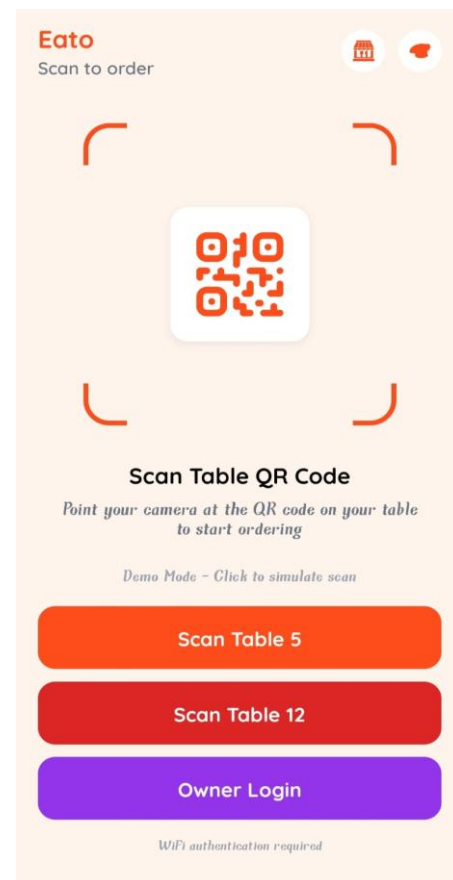


Figure 58 Front Page

8.1.2 Browsing the Menu

Customers can view all available food items, organized into categories, or search for a specific dish.

Steps:

1. Navigate to the Menu tab from the bottom navigation bar.
2. Browse categories displayed as cards.
3. Tap on a category to view all items in that category.
4. Use the search bar to find a specific dish.
5. Tap on an item to view details, price, and description.

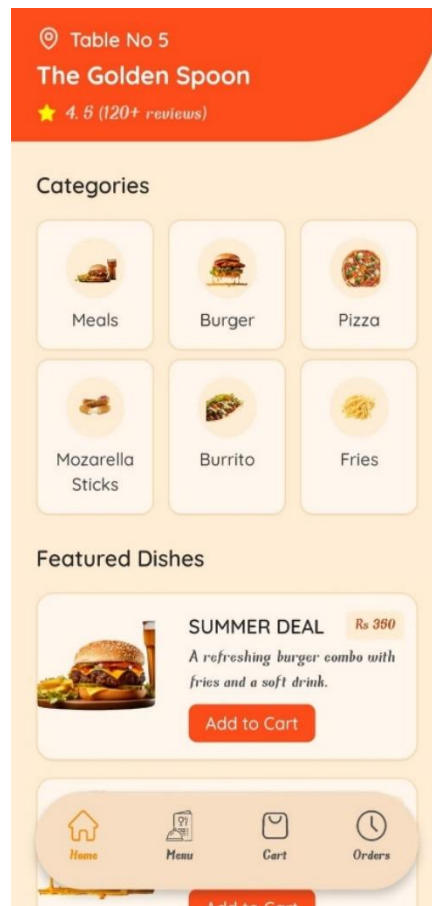


Figure 59 Home page

8.1.3 Adding Items to Cart

Customers can select the quantity and add items to the cart for checkout.

Steps:

1. On the item detail page, adjust the quantity using the plus/minus buttons.
2. Tap Add to Cart.
3. Repeat the process to add more items.

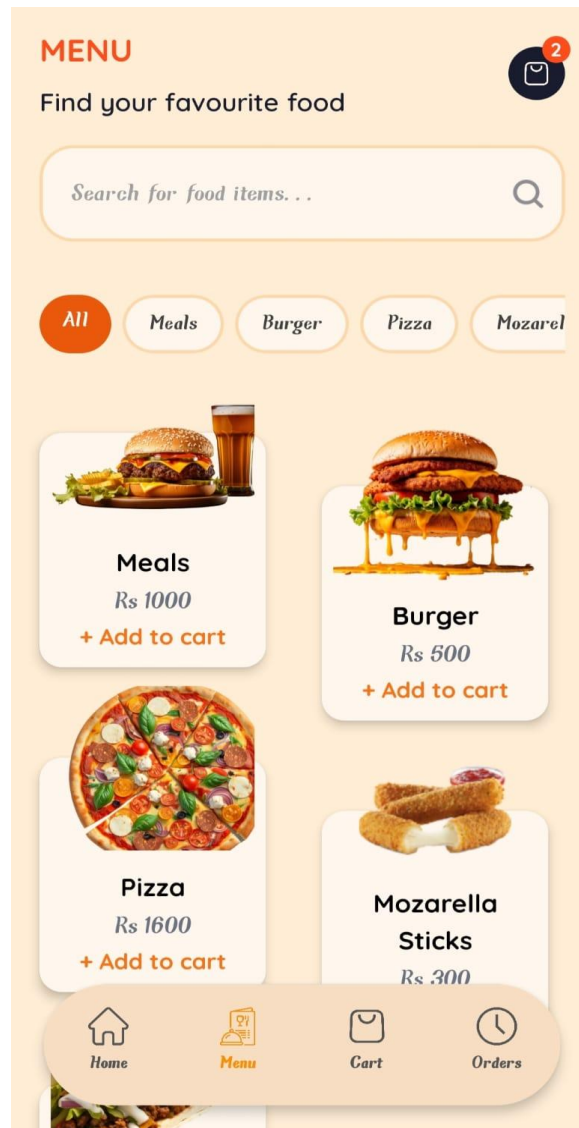


Figure 60 Customer Menu

8.1.4 Viewing and Confirming the Cart

The cart shows a detailed breakdown of selected items, quantities, taxes, and total amount.

Steps:

1. Navigate to the Cart tab.
2. Review all selected items and quantities.
3. Edit quantities or remove items if necessary.
4. Tap Confirm Order to place the order.

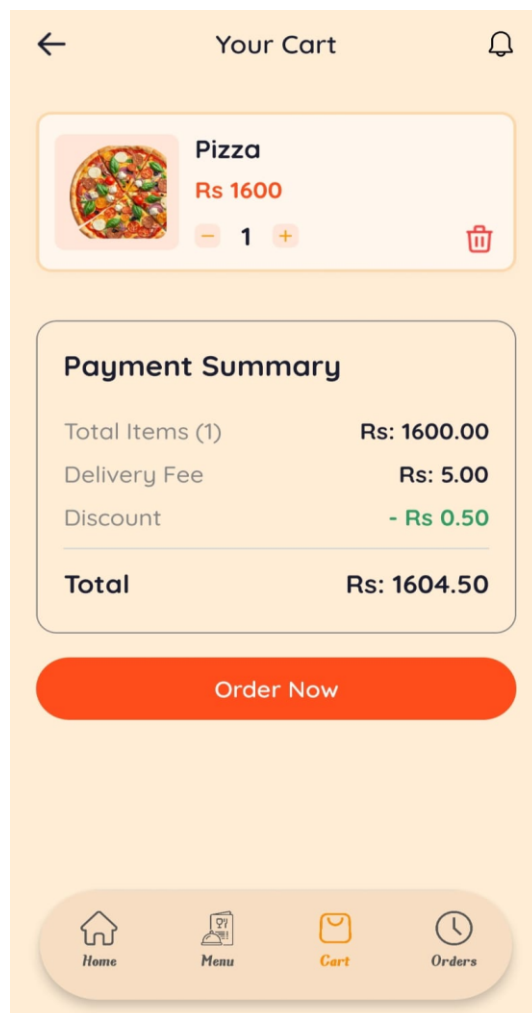


Figure 61 Customer Cart

8.1.5 Tracking Order Status

Customers can track the order in real-time, including preparation status and estimated time.

Steps:

1. Navigate to the Order Status tab.
2. View the current stage: Preparing, Ready, or Served.
3. Estimated preparation time is displayed.
4. Wait for notifications or updates from the restaurant.



Figure 62 Customer Orders Section

8.2 Kitchen Panel

The Kitchen Panel is designed specifically for kitchen staff (chefs) to manage and process orders efficiently. It allows chefs to view incoming orders, update preparation times, and mark orders as ready for serving, ensuring smooth coordination between the kitchen and the dining area.

8.2.1 Login for Kitchen Staff

Kitchen staff can securely log in using credentials provided by the restaurant owner. This ensures that only authorized personnel can access the panel.

Steps:

1. Open the kitchen panel login page on the device or computer provided.
2. Enter the username and password given by the restaurant owner.
3. Click Login.
4. If the credentials are correct, you will be redirected to the Chef Dashboard.

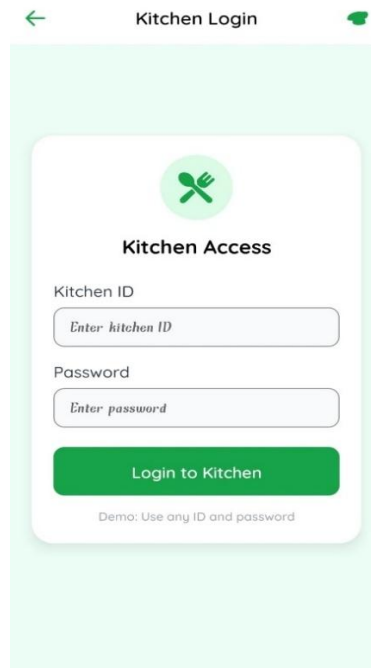


Figure 63 Chef Login

8.3.2 Chef Dashboard

Once logged in, the chef can efficiently manage and process incoming orders, ensuring timely preparation and smooth kitchen operations.

Steps:

1. View all incoming orders in a list format, including details such as items ordered, table number, and any special instructions.
2. Accept or reject orders based on kitchen capacity or ingredient availability.
3. For accepted orders, update the preparation time to indicate when the order will be ready.
4. Once the order is prepared, mark it as Ready to notify the serving staff.
5. Monitor pending and completed orders to organize kitchen workflow efficiently.



Figure 64 Chef Dashboard

8.3 Setting Up a New Restaurant

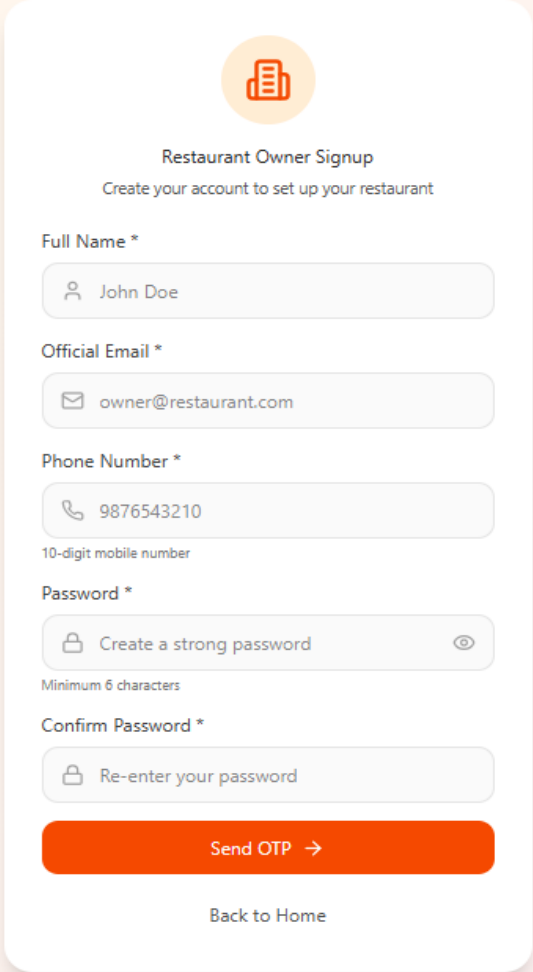
This section guides restaurant owners through the initial setup process for their restaurant on the Eato platform. The setup consists of six pages/screens, including account creation, verification, and configuring the restaurant details, categories, menu, and tables.

5.3.1 Owner Signup

Restaurant owners must first create an account to access the Eato dashboard.

Steps:

1. Open the Eato web or mobile application and navigate to the Owner Signup page.
2. Enter the required details, including:
 - Full Name
 - Email Address
 - Phone Number
 - Password
 - Confirm Password
3. Agree to the terms and conditions.
4. Click Sign Up to proceed.



The screenshot displays the 'Restaurant Owner Signup' form. At the top, there is an orange circular icon with a white building symbol. Below the icon, the title 'Restaurant Owner Signup' is centered, followed by the subtitle 'Create your account to set up your restaurant'. The form contains five input fields, each with a label and an asterisk indicating it is required: 'Full Name *' with a person icon, 'Official Email *' with an envelope icon, 'Phone Number *' with a phone icon, 'Password *' with a lock icon, and 'Confirm Password *' with a lock icon. Each field has a placeholder text: 'John Doe', 'owner@restaurant.com', '9876543210', 'Create a strong password', and 'Re-enter your password' respectively. Below the 'Phone Number' field, there is a small text label '10-digit mobile number'. Below the 'Password' field, there is a small text label 'Minimum 6 characters'. At the bottom of the form, there is an orange button labeled 'Send OTP →' and a link labeled 'Back to Home'.

Figure 65 Restaurant Setup

8.3.2 OTP Verification

After signing up, the owner must verify their phone number to ensure account security.

Steps:

1. An OTP (One-Time Password) will be sent to the registered email address.
2. Enter the OTP in the provided field.
3. Click Verify to confirm the account.
4. Once verified, the owner is redirected to the restaurant setup page.

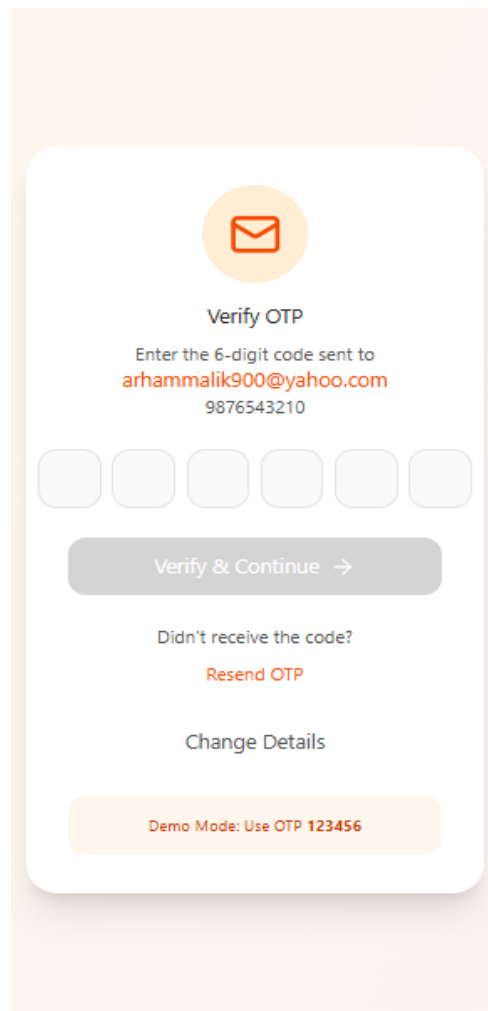


Figure 66 OTP Verification

8.3.3 Enter Restaurant Details

Owners provide the basic information about their restaurant.

Steps:

1. Enter the restaurant name as it should appear in the app.
2. Click Next to continue.

← Restaurant Setup

1 2 3 4

✓ Owner Verified

Owner: Arham Saleem
Email: arhammalik900@yahoo.com
Phone: 9876543211

Restaurant Name

The Golden Spoon

Continue to Categories

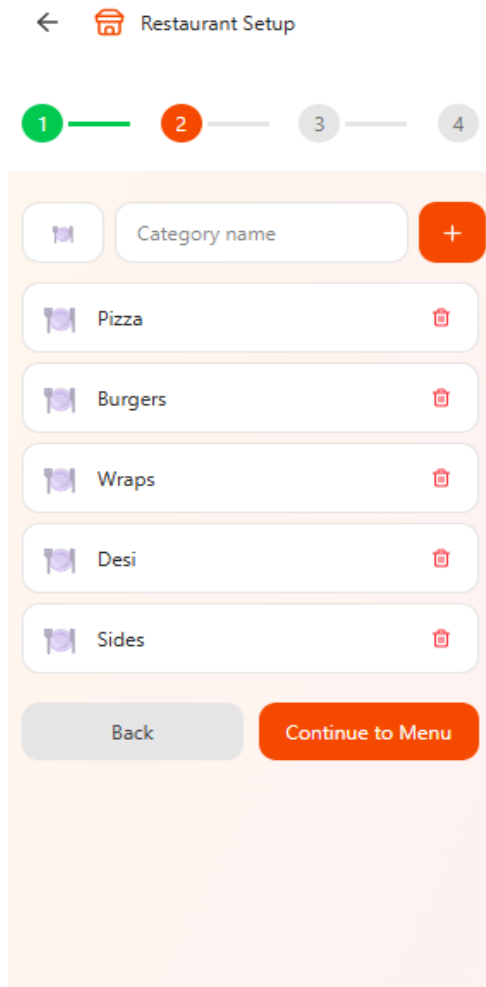
Figure 67 Restaurant Details

8.3.4 Add Categories

Owners can organize their menu into categories to make it easier for customers to browse.

Steps:

1. Click Add Category.
2. Enter the category name (e.g., Appetizers, Main Course, Beverages).
3. Repeat to add multiple categories.
4. Click Next when done.



The screenshot displays the 'Restaurant Setup' interface. At the top, there is a back arrow and the title 'Restaurant Setup'. Below the title is a progress indicator with four steps: 1 (green, active), 2 (orange), 3 (grey), and 4 (grey). The main content area is a light orange box containing a form for adding categories. The form has a header with a purple fork and knife icon, a text input field labeled 'Category name', and an orange '+' button. Below the header are five category entries, each with a purple fork and knife icon, a text field, and a red trash icon. The entries are 'Pizza', 'Burgers', 'Wraps', 'Desi', and 'Sides'. At the bottom of the form are two buttons: a grey 'Back' button and an orange 'Continue to Menu' button.

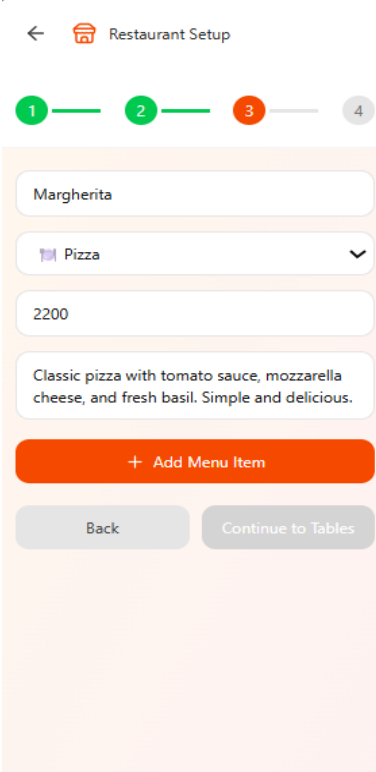
Figure 68 Add Categories

8.3.5 Set Menu

Owners add individual dishes to the menu and assign them to categories.

Steps:

1. Click Add Dish.
2. Enter the dish name.
3. Select the category the dish belongs to.
4. Enter the price of the dish.
5. Add a description for the dish (optional but recommended).
6. Repeat for all menu items.
7. Click Next when the menu is complete.



The screenshot displays a mobile application interface for 'Restaurant Setup'. At the top, there is a back arrow and the title 'Restaurant Setup'. Below the title is a progress indicator with four steps: 1 (green), 2 (green), 3 (red), and 4 (grey). The main form area is a light orange rectangle containing the following elements: a text input field with 'Margherita', a dropdown menu showing 'Pizza' with a downward arrow, a text input field with '2200', a text area with the description 'Classic pizza with tomato sauce, mozzarella cheese, and fresh basil. Simple and delicious.', an orange button with a plus icon and the text '+ Add Menu Item', and two grey buttons at the bottom labeled 'Back' and 'Continue to Tables'.

Figure 69 Add Menu Items

8.3.6 Enter Number of Tables

Owners define the number of tables in their restaurant for QR code generation.

Steps:

1. Enter the total number of tables available in the restaurant.
2. The system will automatically generate unique QR codes for each table.
3. Click Finish to complete the setup.
4. Owners can download and print the QR codes to place on the tables.

← Restaurant Setup

1 2 3 4

Number of Tables

3

Generated QR Codes [Download All](#)

Table 1 Table 2 Table 3

Back Generate QR Codes

Complete Setup

Figure 70 Get QR-Codes

8.4 Owner Login and Dashboard

This section describes how restaurant owners can log in to the Eato platform and manage their restaurant using the Owner Dashboard. The dashboard consists of four main tabs: Owner Dashboard, Orders, Menu, and Profile.

8.4.1 Logging in as Owner

Owners must log in to access their restaurant management features.

Steps:

1. Open the Eato web application and navigate to the Owner Login page.
2. Enter your registered email/username and password.
3. Click Login.
4. Upon successful login, you will be redirected to the Owner Dashboard.

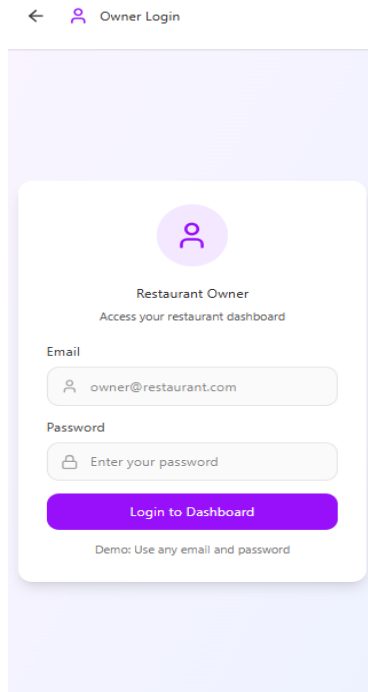


Figure 71 Owner Login

8.4.2 Owner Dashboard Tab

The Owner Dashboard provides a quick overview of restaurant performance and operations.

Features:

1. **Today's Sale:** Displays total revenue for the current day.
2. **Top Selling Items:** Shows the most popular menu items based on current sales.
3. **Total Orders:** Number of all orders placed today.
4. **Pending Orders:** Orders that are yet to be prepared or served.
5. **Served Orders:** Orders that have been completed and served to customers.
6. **Table Requests:** Shows which tables are calling for a waiter.

Steps to Use:

1. Navigate to the Dashboard tab after login.
2. Review all key metrics at a glance.
3. Monitor pending orders and table requests for quick action.

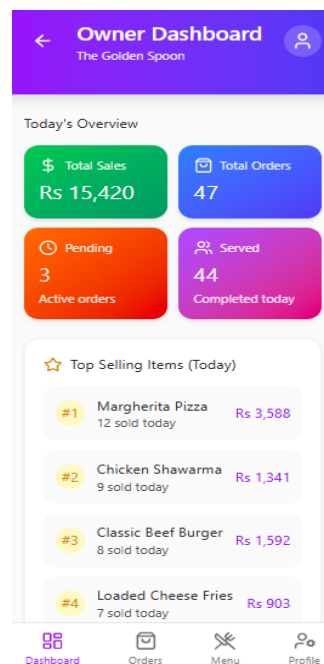


Figure 72 Owner Dashboard

8.4.3 Orders Tab

The Orders tab allows owners to manage and track all restaurant orders.

Features:

1. View Pending Orders and Served Orders.
2. Each order displays table number, bill amount, and item details.
3. Track order status in real-time.

Steps to Use:

1. Click on the Orders tab.
2. Browse the list of pending orders to ensure timely processing.
3. Check served orders for record-keeping and analytics.
4. Click on any order to see detailed items and bill information.

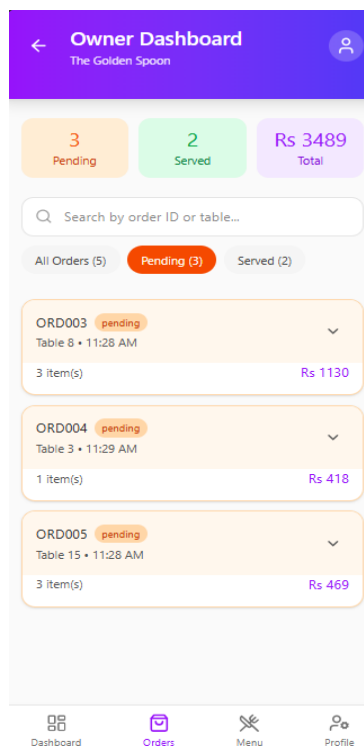


Figure 73 Owner Orders View

8.4.4 Menu Tab

The Menu tab allows owners to fully customize the restaurant menu.

Features:

1. Add, edit, or remove categories.
2. Add, edit, or remove menu items.
3. Adjust prices of items.
4. Update item descriptions and images.

Steps to Use:

1. Navigate to the Menu tab.
2. To add a new category, click Add Category and enter the name.
3. To remove a category, click Delete Category.
4. To add a new dish, click Add Item.
5. Edit existing items to change name, description, price, or image.
6. Save changes to update the menu instantly.

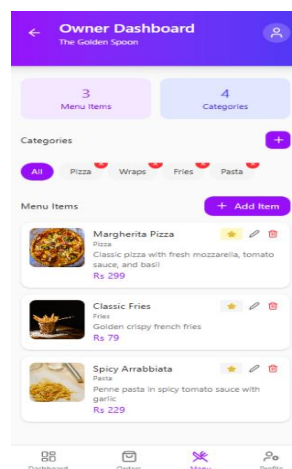


Figure 74 Update Menu

8.4.5 Profile Tab

The Profile tab allows owners to manage restaurant details and user credentials.

Features:

1. Edit restaurant name.
2. Update account password.
3. Manage chef IDs and passwords for kitchen staff access.

Steps to Use:

1. Navigate to the Profile tab.
2. Edit the restaurant name if required.
3. Update your login password for security.
4. Add, edit, or remove chef accounts to control kitchen access.
5. Save changes to update all details.

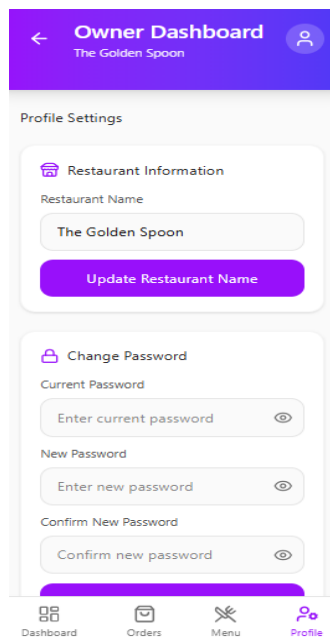


Figure 75 Update Profile

Chapter 9: Lessons Learnt and Future Work

9.1 Lessons Learnt

The development of the Eato system provided valuable technical and professional learning experiences throughout the software development lifecycle. The project involved requirement analysis, system design, implementation, testing, deployment, and maintenance, allowing practical application of concepts learned during academic studies.

One of the most important lessons learned was the significance of proper project planning and requirement gathering. Clearly defining system requirements at the beginning of the project helped reduce development issues and ensured that the final system met user expectations.

The project also enhanced knowledge of full-stack mobile application development through React Native, Express.js, Node.js, and MongoDB Atlas. Practical experience was gained in building scalable applications that integrate frontend and backend components effectively.

The implementation of JWT authentication provided hands-on experience in securing web and mobile applications. The use of Socket.io introduced real-time communication concepts and demonstrated the importance of event-driven systems in modern applications.

Cloud-based technologies such as MongoDB Atlas, Cloudinary, and Render provided practical exposure to cloud computing environments. The project also improved problem-solving, debugging, teamwork, communication, and documentation skills.

Overall, the Eato project served as an excellent opportunity to apply theoretical knowledge in a real-world environment and gain experience in developing a complete software solution from conception to deployment.

9.2 Future Work

Although the Eato system successfully achieves its primary objectives, several enhancements can be implemented in future versions to further improve functionality, usability, and scalability.

Potential future enhancements include:

- Integration of online payment gateways.
- Push notification services.

- Multi-restaurant support on a larger scale.
- Advanced analytics and reporting dashboards.
- Customize customer panel according to restaurant theme.
- Customer feedback and rating systems.
- Enhanced owner and chef management applications.
- Provide web page for owner dashboard.

These improvements have the potential to transform Eato into a comprehensive restaurant management ecosystem capable of serving a wider range of business requirements and providing an enhanced user experience.

References

- [1] M. Multazam and S. Erniwati, 'DEVELOPMENT AND USABILITY EVALUATION OF A MOBILE WEB-BASED RESTAURANT SYSTEM FOR DIGITAL ORDERING AT RESTO DASKER', *J. Kecerdasan Buatan Dan Teknol. Inf.*, vol. 5, no. 2, pp. 328–336, May 2026, doi: 10.69916/jkbt.v5i2.492.
- [2] J. Mate, 'Strategic Pathways for Digital Transformation in SMEs: Challenges and Opportunities', Jan. 2026.
- [3] B. Akhil, 'Smart Restaurant Management System Using QR Code', *Int. J. Res. Appl. Sci. Eng. Technol.*, vol. 14, pp. 4888–4889, Mar. 2026, doi: 10.22214/ijraset.2026.78015.
- [4] S. Unnikrishnan, 'A Study on Cloud Kitchens and Their Impact on Restaurants in Coimbatore', Mar. 2026, doi: 10.5281/zenodo.19032598.
- [5] A. Saxena and N. Gupta, 'An Analysis Of Customer Preference Towards Digital Food Ordering Applications In Uttar Pradesh', *Int. J. Sci. Technol. Res.*, vol. 9, pp. 1131–1134, Apr. 2020.
- [6] A. Santoso, N. Maulana, A. Zagladi, and E. Nora, 'Digital Transformation Strategies for Human Resources: A Literature Review', *Formosa J. Multidiscip. Res.*, vol. 5, pp. 1115–1124, Apr. 2026, doi: 10.55927/fjmr.v5i4.47.
- [7] 'Online Ordering System for Restaurants', UpMenu. Accessed: Jun. 04, 2026. [Online]. Available: <https://www.upmenu.com/>
- [8] 'react-native-qrcode-generator', npm. Accessed: Jun. 04, 2026. [Online]. Available: <https://www.npmjs.com/package/react-native-qrcode-generator>
- [9] M. Al-Shareeda, '2025 Reviewer Certificate from The IEEE Computer Society'. Jan. 21, 2026.
- [10] 'Introduction | Socket.IO'. Accessed: Jun. 04, 2026. [Online]. Available: <https://socket.io/docs/v4/>
- [11] D. Ferraiolo and D. Kuhn, 'Role-Based Access Controls', Mar. 2009.
- [12] Auth0, 'JSON Web Token Introduction - jwt.io', JSON Web Tokens - jwt.io. Accessed: Jun. 04, 2026. [Online]. Available: <https://jwt.io/introduction>

- [13] S. Peldszus *et al.*, *Developer Perspectives on REST API Usability: A Study of REST API Guidelines*. 2026. doi: 10.48550/arXiv.2601.16705.
- [14] T. Ruas and W. Grosky, *Software Engineering: A Practitioner's Approach 9 th Edition*. 2020.
- [15] 'Accelerating Full-Stack Development with the MERN Technology Stack', Jul. 2025.
- [16] 'Hello World Class Component - Snack'. Accessed: Jun. 04, 2026. [Online]. Available: <https://snack.expo.dev>
- [17] 'Documentation for Visual Studio Code'. Accessed: Jun. 04, 2026. [Online]. Available: <https://code.visualstudio.com/docs>
- [18] 'Android Essentials | Android Developers'. Accessed: Jun. 04, 2026. [Online]. Available: https://developer.android.com/get-started?gclsrc=aw.ds&gad_source=1&gad_campaignid=22798875564&gbraid=0AAAAAIOZnhTz88hO3rHPkBWmWILbdZ1&gclid=CjwKCAjwxITRBhBYEiwA6mZm7ej2StV4HXS9i9XK6KUDZo_YYVgg68XUzNqvV_dXjME006camaD1zFBoCGg0QAvD_BwE
- [19] 'MongoDB Atlas: Multi-Cloud Database Service - Atlas - MongoDB Docs'. Accessed: May 31, 2026. [Online]. Available: <https://www.mongodb.com/docs/atlas/>
- [20] 'Docs + Quickstarts | Render'. Accessed: Jun. 04, 2026. [Online]. Available: <https://render.com/docs>
- [21] 'Expo Documentation'. Accessed: Jun. 04, 2026. [Online]. Available: <https://docs.expo.dev/>
- [22] 'Cloudinary Image & Video API Platform - Docs Home Page | Documentation'. Accessed: May 31, 2026. [Online]. Available: <https://cloudinary.com/documentation>
- [23] 'Express.js · Node.js web application framework', Express.js. Accessed: May 31, 2026. [Online]. Available: <https://expressjs.com/en/>
- [24] 'First steps | axios | Promise based HTTP client'. Accessed: Jun. 04, 2026. [Online]. Available: <https://axios.rest/pages/getting-started/first-steps>
- [25] 'Redux Toolkit'. Accessed: Jun. 04, 2026. [Online]. Available: <https://redux-toolkit.js.org/>

Appendix

Appendix A: Detailed User Role Descriptions

Customer

- Scans table-specific QR code
- Accesses digital menu
- Searches and filters menu items
- Manages cart and places orders
- Tracks order status in real time

Restaurant Staff (Management)

- Manages restaurant profile and branding
- Adds, edits, and deactivates menu items
- Generates and manages table QR codes
- Views all active and completed orders
- Monitors basic sales analytics

Kitchen Staff (Chef)

- Receives incoming orders in real time
- Views detailed order information
- Accepts or rejects orders
- Sets preparation time
- Updates order status (Preparing, Ready)

Appendix B: Detailed Order Status Flow

- **Received** – Order successfully submitted by customer
- **Accepted** – Order acknowledged by kitchen staff
- **Preparing** – Food preparation in progress
- **Ready** – Order prepared and ready for pickup/serving
- **Served** – Order delivered to the table

Appendix C: Alternate and Exception Handling Scenarios

- Manual table code entry if QR scanning fails

- Notification to customer if an item becomes unavailable after selection
- Prevention of order placement without restaurant Wi-Fi connection
- Session expiration if the customer leaves the restaurant network

Appendix D: Sample Data Entities (High-Level)

- **Restaurant:** Name, logo, Wi-Fi ID, table count
- **Table:** Table ID, QR code, status
- **Menu Category:** Category ID, name, active status
- **Menu Item:** Item ID, name, description, price, availability
- **Order:** Order ID, table ID, item list, status, timestamp

Appendix E: System Constraints and Assumptions

- Customers must be physically present inside the restaurant
- System is designed for dine-in ordering
- Analytics provided are basic and intended for small-scale businesses

Appendix F: Academic and Practical Relevance

- Demonstrates real-world application of mobile app development
- Supports concepts of role-based access control
- Implements real-time data synchronization
- Suitable for both academic evaluation and commercial scalability